

Mobile & CPS — Modulo Chessa (Riassunto)

Fabio Piscitelli

2026

Indice

1	01 - Fondamenti IoT e Progettazione	1
1.1	1. Introduzione ai Cyber-Physical Systems e IoT	1
1.2	2. Piattaforme IoT, Astrazione e Gestione dei Dati	2
1.3	3. Architetture di Rete: Edge, Fog e Cloud	3
1.4	4. Intelligenza Artificiale, Machine Learning e Blockchain	4
1.5	5. Interoperabilità e Standard	5
1.6	6. Sicurezza nell'IoT	6
1.7	7. IoT Design	6
1.8	8. Case Study: Biologging e Activity Recognition	9
1.9	9. Embedded Programming e Arduino	16
1.10	10. Domande d'esame frequenti	19
2	02 - Protocolli Applicativi (MQTT e CoAP)	20
2.1	I Requisiti dell'IoT e la Conventional Internet Protocol Suite	20
2.2	Introduzione al Protocollo MQTT	21
2.3	Il Paradigma Publish / Subscribe	21
2.4	Il Modello MQTT: Connessione e Flusso Operativo	21
2.5	I Meccanismi della Quality of Service (QoS)	23
2.6	Meccanismi di Affidabilità Avanzati	23
2.7	Struttura dei Pacchetti MQTT	24
2.8	Implementazione Pratica: MQTT su Arduino	25
2.9	MQTT vs HTTP e il Problema della Scalabilità	25
2.10	CoAP: L'Alternativa per Reti Vincolate	25
3	Livello MAC e Reti WSN	26
3.1	1. Protocolli MAC per Reti Wireless Multi-hop (Lezione 17)	26
3.2	2. Lo Standard IEEE 802.15.4 (Lezione 18)	30
3.3	3. Lo Standard ZigBee: Network Layer (Lezioni 9)	34
3.4	4. Application Layer: ZDO, APS e ZCL (Lezione 12)	38
3.5	5. ZigBee Security (Lezione 13)	39
4	Gestione Energetica: Energy Harvesting e Neutralità	41
4.1	Definizioni fondamentali	41
4.2	Architetture di harvesting	42
4.3	Fonti di energia e classificazione	43
4.4	Misurare la carica e la produzione	44
4.5	Neutralità energetica e Approccio di Kansal	47
4.6	Modello Task-Based per la Neutralità Energetica	50
4.7	Riepilogo	51
5	05 - Preparazione Esame Orale	53
5.1	Interoperabilità	53

5.2	IoT & Security	54
5.3	Publish/Subscribe & MQTT	55
5.4	MQTT – II (QoS 2)	56
5.5	MQTT-III — Topic, Sessioni, Retained Messages, Last Will & Keep Alive	56
5.6	ZigBee - I (Join through Association)	57
5.7	ZigBee – II (Application Layer Stack)	58
5.8	ZigBee - III (Topologie di Rete)	59
5.9	ZigBee - IV (Indirizzamento ad Albero)	60
5.10	ZigBee - V (Tabella APS / Binding Table)	61
5.11	ZigBee - VI (Cluster e Binding tra Dispositivi)	61
5.12	Duty Cycle - I (Calcolo del Duty Cycle)	62
5.13	Duty Cycle - II (Impatto sulla Vita della Batteria)	63
5.14	Embedded Programming - I (Modello Arduino)	64
5.15	Embedded Programming - II (Modello TinyOS/Event-Driven)	64
5.16	Arduino - Interrupt	65
5.17	SMAC (Sensor-MAC)	66
5.18	BMAC - I (Struttura del Protocollo)	66
5.19	BMAC - II (Trade-off t_{check} e Vita del Trasmettitore)	68
5.20	X-MAC / B-MAC (Confronto LPL vs X-MAC)	68
5.21	IEEE 802.15.4 - I (Struttura del Superframe)	69
5.22	IEEE 802.15.4 - II (Indirect Data Transfer)	70
5.23	IEEE 802.15.4 - III (Protocollo di Associazione)	71
5.24	Energy Harvesting — Domande Generali	71
5.25	Energy Harvesting - I (Grafico P_s vs P_c)	72
5.26	Energy Harvesting – II (Stima Stato di Carica)	73
5.27	Energy Harvesting – III (Approccio Kansal all’Energy Neutrality)	74
5.28	Energy Harvesting – IV (Task Model per l’Energy Neutrality)	75
5.29	Directed Diffusion	76
5.30	GPSR - I (Greedy Forwarding e Void)	77
5.31	GPSR - II (Perimeter Routing / Face Traversal)	77
5.32	GPSR - III (Planarizzazione: Gabriel Graph e RNG)	78

Capitolo 1

01 - Fondamenti IoT e Progettazione

1.1 1. Introduzione ai Cyber-Physical Systems e IoT

I sistemi cyber-fisici (**Cyber-Physical Systems**, CPS) vivono e connettono due mondi distinti: possiedono sensori e attuatori per interagire con il mondo fisico e, contemporaneamente, sono entità cibernetiche dotate di capacità di elaborazione, memoria e comunicazione per agire nel cberspazio. L'**Internet of Things** (IoT) rappresenta una “incarnazione” concreta di questi sistemi, implementando quelli che vengono definiti **ambienti intelligenti** (*smart environments*).

Definizione – Cyber-Physical System (CPS)

Sistema che integra capacità computazionali e di comunicazione con il controllo diretto del mondo fisico tramite sensori e attuatori. La componente cibernetica e quella fisica sono inseparabili e si influenzano reciprocamente.

In un ambiente intelligente, gli oggetti (*smart objects*) assumono una duplice natura fisica e cibernetica. Dal punto di vista fisico, qualsiasi oggetto è soggetto a esperienze tangibili: può essere posizionato in luoghi diversi, spostato ed è esposto ai rischi dell’ambiente esterno, come danni, furti o manomissioni. Questi dispositivi sono caratterizzati da posizionamento libero, dimensioni ridotte, diverse forme (*form factor*) e gusci protettivi. La mobilità è garantita da capacità di comunicazione wireless e alimentazione a batteria, rendendoli spesso consapevoli della propria posizione (*position-aware*). L’esposizione all’ambiente esterno impone ridondanza, sicurezza e costi contenuti.

Un ambiente si definisce “smart” principalmente in relazione all’utente finale: è in grado di riconoscere il contesto, le attività e le situazioni, comprendendo i bisogni dell’utente al momento giusto e fornendo servizi — anche fisici — tramite attuatori o robot. Le caratteristiche comuni di questi ambienti includono la pervasività e la natura non intrusiva dei dispositivi cyber-fisici. Il concetto di “smart” si estende però anche alla gestione, al dispiegamento (*deployment*) e alla manutenzione del sistema stesso: un sistema intelligente deve essere sostenibile, flessibile, sicuro e facile da usare.

1.1.1 Le Quattro Generazioni di Internet

L’IoT rappresenta l’ultimo sviluppo di Internet e del computing, interconnettendo dispositivi intelligenti che vanno dagli elettrodomestici ai sensori minuscoli, integrando ricetrasmittitori mobili negli oggetti di uso quotidiano. Internet supporta la loro connettività, solitamente verso sistemi cloud, abilitando nuove forme di comunicazione tra persone e cose, e tra le cose stesse. Gli oggetti forniscono informazioni sensoriali, agiscono sul loro ambiente e, in alcuni casi, modificano se stessi come parte di un sistema più ampio.

È possibile identificare quattro generazioni di Internet in base ai sistemi finali supportati:

Generazione	Tecnologia	Dispositivi tipici	Caratteristiche
Information Technology (IT)	Cablata	PC, server	Elaborazione e storage centralizzati
Operational Technology (OT)	Cablata/industriale	Macchinari, sistemi di controllo	Automazione di processi industriali
Personal Technology	Wireless	Smartphone, tablet	Connettività mobile personale
Sensor/Actuator Technology (IoT)	Wireless, batteria	Sensori, attuatori monoscopo	Bassa potenza, bassa banda, sistemi embedded

L'IoT è guidato principalmente da dispositivi embedded a bassa potenza, bassa larghezza di banda ed energia limitata, ma include anche apparecchi come telecamere di sicurezza ad alta risoluzione che richiedono elevate capacità di streaming.

1.1.2 Architettura a Livelli dell'IoT

Ogni dispositivo IoT è composto da sensori e attuatori, un microcontrollore, un'interfaccia wireless e un software che contiene la logica di business. L'architettura IoT è strutturata a livelli sovrapposti che rispecchiano la separazione tra mondo fisico, rete e applicazioni.

Alla base si trova il livello di **Percezione**, costituito dagli oggetti fisici e dai sensori che raccolgono dati dall'ambiente. Sopra di esso si trova il livello di **Comunicazione**, che gestisce le tecnologie di rete e wireless responsabili del trasporto dei dati. Seguono il livello di gestione delle risorse e dei servizi e il livello di gestione dei dati e della conoscenza, che comprende storage e *data analytics*. Al vertice vi sono i servizi specifici del dominio applicativo, come Smart Industry, Smart Energy o Smart Transport. Parallelamente a tutti i livelli opera la sicurezza, che non è un componente isolato ma una proprietà trasversale all'intera architettura.

Nota – Sensori al livello di percezione

I sensori possono monitorare parametri molto diversi a seconda del contesto. Per il tracciamento degli asset si utilizzano geolocalizzazione GNSS, prossimità NFC o rilevamento dell'orientamento tramite giroscopi. Per la logistica di magazzino si impiegano tecnologie come il Time of Flight (UWB) o l'Angle of Arrival (Bluetooth) per la localizzazione precisa al chiuso.

1.2 2. Piattaforme IoT, Astrazione e Gestione dei Dati

Le **piattaforme IoT** agiscono come strati software tra i dispositivi e le applicazioni, distribuendo le funzionalità tra i dispositivi stessi, i gateway e i server nel cloud. Queste piattaforme offrono un insieme di funzionalità critiche che coprono l'intero ciclo di vita dei dispositivi e dei dati.

1.2.1 Identificazione, Discovery e Device Management

L'**identificazione** fornisce un metodo univoco per riconoscere le entità all'interno di una piattaforma. Esistono diversi standard a seconda del contesto operativo: gli indirizzi IP per Internet, l'**MSISDN** nella telefonia, gli **URI** sul web, l'**OID** nelle telecomunicazioni e gli **UUID** nei sistemi informatici (molto utilizzati anche nello standard Bluetooth).

La **discovery** è il meccanismo attraverso il quale la piattaforma individua dispositivi, risorse o servizi all'interno di un'infrastruttura IoT, permettendo di interrogarne in modo dinamico le proprietà e le caratteristiche offerte.

La fase di **gestione dei dispositivi** (Device Management) parte dall'inizializzazione e configurazione, attività che includono il pairing, la definizione delle impostazioni di sicurezza con la relativa distribuzione delle chiavi

crittografiche, la calibrazione dei trasduttori e la localizzazione spaziale degli endpoint. La piattaforma IoT si occupa anche di gestire l'aggiornamento automatico del software e del firmware, di monitorare in tempo reale lo stato dell'hardware (come il livello batteria o la temperatura) e di effettuare diagnostica per guasti. Questi processi si appoggiano spesso su standard come **OMA DM** e **OMA LWM2M** per terminali e sensori, e **BBF TR-069** per apparecchiature destinate agli utenti finali.

1.2.2 Astrazione, Semantica e Composizione dei Servizi

L'**astrazione** e la **virtualizzazione** nascondono la complessità fisica dell'hardware per trattare i dispositivi IoT come veri e propri servizi, introducendo concetti architetturali chiave come il **digital twin** per l'Industry 4.0.

Definizione – Digital Twin

Il digital twin è una rappresentazione virtuale di un oggetto o sistema fisico, mantenuta sincronizzata con la sua controparte reale. Nell'Industry 4.0 permette di simulare, monitorare e ottimizzare il comportamento dei dispositivi fisici senza interagire direttamente con essi.

La **semantica** attribuisce significato strutturato ai dati raccolti, permettendo di rappresentare il contesto operativo degli apparati per abilitare un ragionamento logico e favorire l'elaborazione dei flussi informativi da parte dell'intelligenza artificiale.

La **service composition** si incarica di aggregare i servizi di svariati dispositivi IoT e di componenti software preesistenti per dare origine a funzionalità composite o catene di data analytics complesse.

1.2.3 Database NoSQL e MongoDB

I flussi informativi ad alta intensità tipici dell'IoT richiedono soluzioni di archiviazione flessibili e performanti, motivo per cui ci si affida sovente ai database **NoSQL**, pensati per applicazioni real-time e scenari Big Data. I sistemi NoSQL offrono prestazioni eccellenti scalando in modo orizzontale su cluster di macchine e hanno un design più snello e destrutturato rispetto all'approccio relazionale SQL classico.

MongoDB, ad esempio, salva le proprie entità non come record tabellari ma come documenti formattati in una sintassi molto vicina al JSON, potendo incorporare array e oggetti nidificati per ridurre l'esecuzione di operazioni costose come le join. I documenti vengono logicamente raggruppati in **collection** — assimilabili alle tabelle — pur non dovendo condividere necessariamente la medesima struttura rigida. Le query su un database MongoDB consentono l'applicazione flessibile di criteri di filtraggio, l'utilizzo di proiezioni e modificatori per l'ordinamento.

1.3 3. Architetture di Rete: Edge, Fog e Cloud

Le problematiche di latenza, affidabilità e larghezza di banda nell'IoT vengono affrontate distribuendo l'elaborazione su diversi livelli di rete, ciascuno con caratteristiche e responsabilità distinte.

Definizione – Cloud

Livello centrale dell'architettura, caratterizzato da data center cablati e tempi di risposta transazionali. Adatto per l'archiviazione a lungo termine e l'elaborazione computazionalmente pesante.

Definizione – Edge

Livello periferico della rete aziendale, costituito dai dispositivi IoT stessi (sensori e attuatori) o dai gateway che li interconnettono. Opera con tempi di risposta nell'ordine dei millisecondi, offrendo la possibilità di azioni locali real-time.

Definizione – Fog Computing

Livello intermedio tra Edge e Cloud. I dispositivi Fog sono fisicamente vicini all'Edge e convertono i flussi di dati di rete in informazioni adatte all'archiviazione o all'elaborazione di alto livello, riducendo il volume di dati inviati al cloud e garantendo tempi di risposta in tempo reale.

Lo scopo del Fog è eseguire l'elaborazione il più vicino possibile ai sensori — valutazione, formattazione, riduzione dei dati — alleggerendo così il traffico verso il cloud e abbattendo la latenza per le applicazioni che richiedono risposte immediate.

Tip – Intuizione chiave

La scelta tra Edge, Fog e Cloud non è esclusiva: le architetture IoT moderne sfruttano tutti e tre i livelli in modo complementare, assegnando a ciascuno il tipo di elaborazione più adatto alle sue caratteristiche.

1.4 4. Intelligenza Artificiale, Machine Learning e Blockchain

L'**Intelligenza Artificiale** (AI) nell'IoT mira a rendere i sistemi capaci di comportamenti intelligenti, ricercando modelli comportamentali ottimali. Il **Machine Learning** (ML) permette ai sistemi di imparare dai dati (*training set*) per associare input a output, generalizzando poi su dati mai visti prima, e si divide in: - **Unsupervised Learning**: cerca in autonomia pattern nascosti nei dati. - **Supervised Learning**: impara da esempi in cui sono forniti sia l'input sia l'output desiderato. - **Reinforcement Learning**: evolve attraverso un meccanismo di rinforzo o ricompensa.

Nota – TinyML

Il paradigma **TinyML** permette di miniaturizzare e implementare modelli di Machine Learning direttamente su processori IoT a bassa potenza, portando capacità inferenziali all'edge senza dipendere da connettività cloud.

1.4.1 Federated Learning

Definizione – Federated Learning

Un'architettura di intelligenza artificiale distribuita che ovvia alla necessità del ML tradizionale di accentrare tutti i dati su un singolo server. Ciascun dispositivo edge addestra autonomamente il proprio modello usando i propri dati in locale. Solo i parametri aggiornati vengono inoltrati a un server di aggregazione che ricompila un nuovo modello globale e lo propaga nuovamente a tutti gli endpoint.

Questo approccio offre enormi vantaggi in scalabilità e privacy. Tuttavia, è soggetto ad attacchi di **poisoning** (avvelenamento): - **Data poisoning**: l'avversario corrompe i dati locali inserendo falle volontarie. - **Model poisoning**: l'avversario manipola i gradienti prima di inviarli all'aggregatore centrale. Per difendersi si applicano metodologie di anomaly detection, reputation systems e auditing tramite Blockchain.

1.4.2 IoT e Blockchain

La **Blockchain** offre un registro pubblico distribuito in cui nessuna entità governa in maniera centralizzata lo storico delle transazioni. Le iscrizioni sono *tamper-evident* (non modificabili) e certificano un unico punto della verità. Uno scenario d'uso frequente è la **supply chain**: reti di sensori analizzano lo stato delle merci per tutte le fasi di transito e firmano le conformità tramite **smart contract** direttamente sul registro distribuito condiviso tra le aziende della catena logistica.

1.5 5. Interoperabilità e Standard

L'**interoperabilità** rappresenta una delle sfide cruciali nello sviluppo dell'Internet of Things. L'assenza di standardizzazione produce architetture in totale isolamento.

Definizione – Vertical Silos

In questo modello, una soluzione funziona esclusivamente all'interno del proprio ecosistema: i dispositivi proprietari comunicano solo con l'infrastruttura dello stesso fornitore, rendendo incompatibili i prodotti di terze parti.

Definizione – Vendor Lock-in

La pratica di “ingabbiare” il cliente con l'obiettivo di prevenire l'utilizzo di componenti di altri produttori e imporre costi elevati per l'eventuale migrazione verso soluzioni alternative.

1.5.1 Tecnologie Wireless e Standard di Riferimento

Il panorama delle tecnologie wireless è differenziato in base al rapporto tra raggio di copertura e velocità di trasmissione: - **IEEE 802.11 (Wi-Fi)**: originariamente a 2.4 GHz (1-2 Mbps), evoluto in bande 5 GHz con velocità di oltre 400 Mbps e forte gestione del Quality of Service (QoS). - **IEEE 802.15.4 e ZigBee**: definiscono i livelli fisico, MAC e applicativi per reti a basso consumo. ZigBee è progettato specificamente per reti di sensori a bassa potenza e basso throughput (fino a 115 Kbps), con duty cycle ridottissimo (~1%). Supporta configurazioni multi-hop. - **Bluetooth e BLE**: offrono data rate superiore, orientati alla comunicazione personale e multimediale a corto raggio. Usa topologia master-slave in *piconet*. - **Reti cellulari**: dal 1G al 4G LTE, con riduzione della latenza sotto i 20 ms.

1.5.2 Il Paradigma 5G

Tip – Il 5G come cambio di paradigma

Il 5G non rappresenta solo un incremento di velocità, ma un cambiamento profondo che abilita categorie di scenari d'uso radicalmente diverse, ciascuna con requisiti tecnici incompatibili tra loro.

Macro-area	Sigla	Applicazione tipica	Requisiti Chiave
Enhanced Mobile Broadband	eMBB	Video HD, realtà virtuale	Altissima velocità, larghezza di banda
Massive Machine Type Communication	mMTC	Smart city, decine di migliaia di sensori	Scala immensa, dispositivi a basso consumo
Ultra-Reliable Low Latency Communications	URLLC	Automazione industriale, guida autonoma	Affidabilità e latenza vitali (non la velocità)

1.5.3 La Necessità degli Standard e i Gateway

Gli standard nascono per ridurre i costi in regime di **coopetition** (cooperazione tra competitor). Tuttavia, la loro proliferazione (MQTT, CoAP, LWM2M) ha spostato le incompatibilità ai livelli middleware e applicativo.

Per gestire questa eterogeneità si ricorre agli **Application-level gateways** o integration gateways. Non si limitano a tradurre pacchetti: mappano comportamenti applicativi differenti l'uno nell'altro, operando come interpreti semantici tra ecosistemi incompatibili.

Tipo	Fornitore	Protocollo	Necessità di gateway
A	Unico	Unico	No
B	Multiplo	Unico (condiviso)	No o minimale
C	Multiplo	Diversi	Sì — Integration Gateway per la traduzione
C/II	Multiplo (consumer)	Diversi	Sì — ecosistemi come Google Home o Alexa
D	Multiplo	Eterogenei e distribuiti	Sì — gateway multipli con mappature complesse

1.6 6. Sicurezza nell'IoT

La sicurezza nei sistemi cyber-fisici ha raggiunto un punto di crisi. A differenza dei sistemi IT tradizionali, i dispositivi IoT sono spesso sistemi embedded economici (**constrained devices**), prodotti con forti incentivi a ridurre i costi (Time-to-Market). Sono frequentemente affetti da vulnerabilità per le quali non esiste meccanismo di patching sistematico, e mostrano debolezze basilari come password hard-coded.

1.6.1 Requisiti di Sicurezza secondo ITU-T Y.2066

La raccomandazione **Y.2066** dell'ITU-T identifica i requisiti fondamentali organizzandoli in tre aree concettuali: 1. **Sicurezza della comunicazione**: garantire riservatezza e integrità dei dati durante la trasmissione tra dispositivi e piattaforme. 2. **Sicurezza della gestione dei dati**: proteggere riservatezza e integrità quando i dati sono archiviati o elaborati (*data at rest*). 3. **Sicurezza della fornitura del servizio**: prevenire accessi non autorizzati ai servizi e proteggere la privacy.

A questi si aggiunge l'implementazione dell'**autenticazione mutua** tra dispositivi (più robusta di quella a una via) e l'auditing tracciabile.

1.6.2 Il Ruolo del Gateway nella Sicurezza

Essendo i dispositivi vincolati spesso incapaci di implementare forte crittografia, il **gateway** agisce come punto centrale di applicazione delle policy: - Gestisce l'identificazione e l'autenticazione di ogni dispositivo connesso. - Assicura la protezione della privacy e decodifica i flussi criptati. - Esegue diagnostica e supporta l'aggiornamento remoto del firmware. - Gestisce configurazioni di policy di sicurezza locali o remote.

1.7 7. IoT Design

1.7.1 Anatomia di un nodo IoT

Un dispositivo IoT tipico è un sistema a basso costo, bassa potenza e di piccole dimensioni. I suoi componenti essenziali sono un **microprocessore** (spesso un MCU da pochi MHz come l'ATmega128L), una piccola quantità di **memoria**, un **ricetrasmittitore radio** (Wireless NIC) e una **scheda sensori** (sensor board) che può misurare accelerazione, pressione, umidità, luce, temperatura, GPS e molto altro. A queste si aggiungono eventuali **attuatori** e una fonte di energia, tipicamente una batteria o celle solari.

Definizione – Nodo IoT (Mote)

Un dispositivo IoT è un sistema embedded autonomo, composto da processore, memoria, radio e sensori, progettato per operare con risorse molto limitate di calcolo, memoria, banda e soprattutto energia.

1.7.2 Principali sfide nel design

Il design di un nodo IoT è vincolato da quattro risorse finite: potenza di calcolo, memoria, capacità della batteria e larghezza di banda. Da questi vincoli discendono le sfide principali: - **Efficienza energetica**: ottimizzare ogni operazione. - **Adattabilità**: protocolli e nodi devono rispondere a condizioni variabili di rete e alimentazione. - **Protocolli a bassa complessità e overhead**: le risorse limitate impongono stack MAC e routing molto leggeri. - **Comunicazione multi-hop**: instradamento necessario tra nodi intermedi. - **Mobilità e pre-processing**: elaborare dati sull'edge riduce i consumi della radio.

1.7.3 La Legge di Moore e l'IoT

Definizione – Legge di Moore

Il numero di transistor che possono essere integrati economicamente in un chip cresce esponenzialmente, raddoppiando circa ogni due anni.

La Legge di Moore non ha un'unica lettura: 1. **Stesse dimensioni, più prestazioni, stesso costo**: PC desktop e server. 2. **Stesso costo, metà dimensioni**: wearable. 3. **Stesse prestazioni, metà costo**: questa è l'applicazione chiave in IoT dove si deployano milioni di nodi e il costo unitario è dominante.

Tip – Intuizione chiave

La Legge di Moore va usata per rendere i nodi IoT **più piccoli e più economici**, non necessariamente più potenti. La sfida del design efficiente rimane, perché la crescita della capacità delle batterie è molto più lenta di quella dei transistor.

1.7.4 Efficienza Energetica: Il problema Intel vs Duracell

C'è un'asimmetria fondamentale: le prestazioni dei processori crescono esponenzialmente, mentre la capacità energetica delle batterie rimane quasi piatta. Più il nodo può "fare", più la batteria diventa il collo di bottiglia.

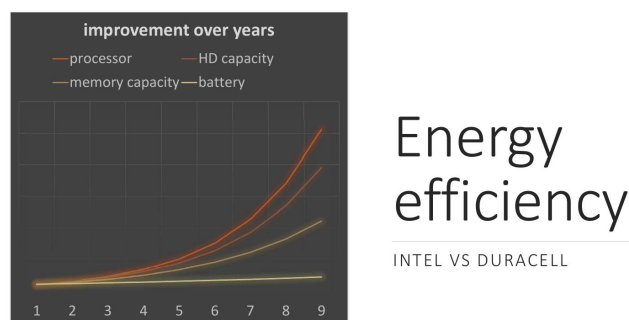


Figura 1.1. Il confronto “Intel vs Duracell”: le curve di processore, HD e memoria crescono esponenzialmente nel tempo, mentre quella della batteria rimane quasi orizzontale. Il gap energetico si allarga progressivamente.

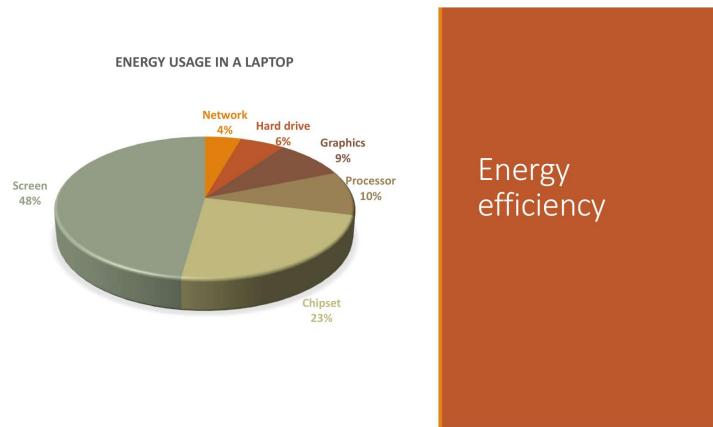


Figura 1.2. Ripartizione del consumo in un laptop: lo schermo domina con il 48%.

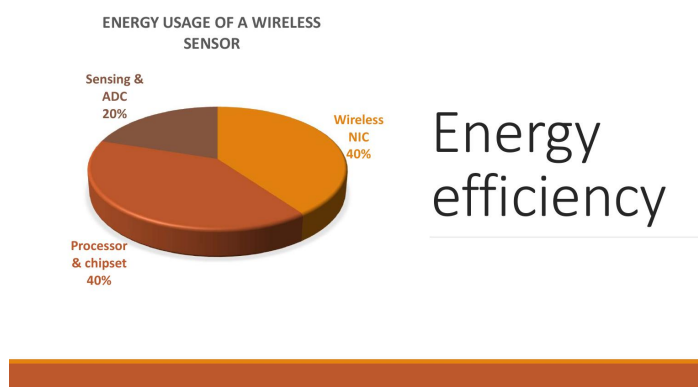


Figura 1.3. Ripartizione del consumo in un sensore wireless: la radio (NIC) e processore pesano il 40% ciascuno, mentre il sensing pesa il 20%. Non essendoci schermo, la radio diventa il componente critico.

1.7.5 Dove va l'energia: laptop vs sensore

Consumi della radio: - Sleep mode: ~10 mA - **Listen mode (in ascolto): ~180 mA** - Receive mode: ~200 mA - Transmit mode: ~280 mA

Attenzione – Radio accesa = energia sprecata

Il consumo in modalità listen è paragonabile a quello in ricezione, e ordini di grandezza superiore allo sleep. La radio deve essere spenta il più possibile. Anche tenere la radio accesa e in attesa è proibitivo.

1.7.6 Duty Cycle

Definizione – Duty Cycle (DC)

Il duty cycle è la frazione di un periodo in cui il sistema (o un suo componente) è attivo: $DC = \frac{t_{attivo}}{T_{periodo}}$.

Applicare duty cycle aggressivi (es. 1-5%) è la leva principale per estendere la vita della batteria, disattivando hardware non necessario in ogni porzione di firmware. Il consumo medio è dettato da $E = dc \cdot C_{active} + (1 - dc) \cdot C_{idle}$.

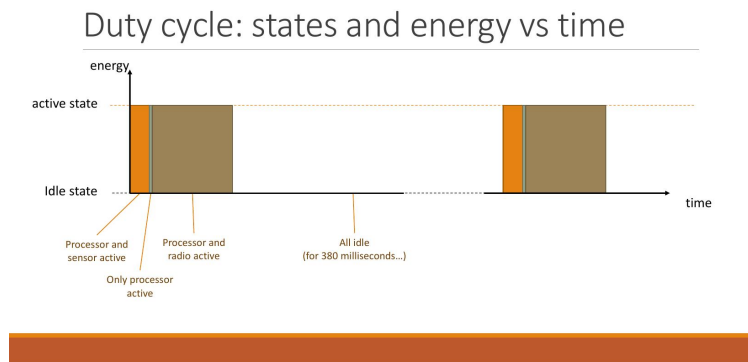


Figura 1.4. Diagramma stati/energia vs tempo per un codice di esempio con turnOn e turnOff dei componenti. Il lungo periodo di idle (380 ms su 400 ms) abbassa drasticamente il consumo medio.

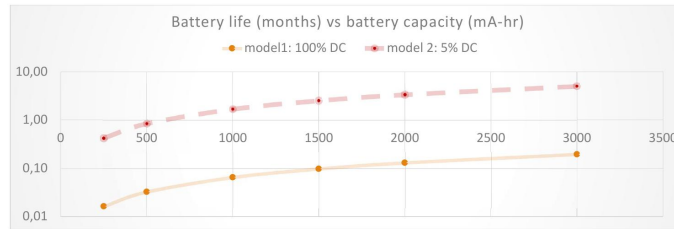
Spegnere la radio tuttavia è una **decisione globale** che richiede protocolli MAC IoT specializzati per permettere la comunicazione sincrona e il risveglio coordinato.

1.8 8. Case Study: Biologging e Activity Recognition

Il biologging rappresenta uno dei casi d'uso più concreti dell'IoT nel mondo naturale. Il progetto **Tortoise@** automatizza il rilevamento delle attività di nidificazione delle tartarughe con ML embedded su dispositivi a bassa potenza.

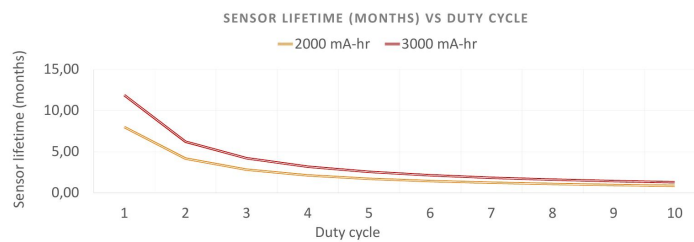
Definizione – Biotelemetria vs Bio-logging

Biotelemetria: i dati raccolti vengono trasmessi in tempo reale. **Bio-logging:** i dati vengono memorizzati internamente per recupero fisico, ed eventualmente trasmessi solo quando richiesto.



Battery life vs DC (I)

Figura 1.5. Vita della batteria vs capacità. Un modello con DC al 5% ha una vita 10-15 volte superiore a uno con DC al 100%.



Battery life vs DC (II)

Figura 1.6. Vita del sensore vs duty cycle: la curva scende molto rapidamente. Ridurre il DC da 3% a 1% genera incrementi molto più incisivi rispetto all'aumentare la batteria da 2000 a 3000 mAh.

	Value	units
Micro Processor (Atmega128L)		
current (full operation)	8	mA
current sleep	15	μA
Radio		
current in receive	19,7	mA
current xmit	17,4	mA
current sleep	20	μA
Logger (storage in the flash memory)		
write	15	mA
read	4	mA
sleep	2	μA
Sensor Board		
current (full operation)	5	mA
current sleep	5	μA
Battery Specifications		
Capacity Loss/Yr	3	%

Specs. of a
mote class-
sensor

Figura 1.7. *Specifiche hardware complete di una Mote-class sensor.*

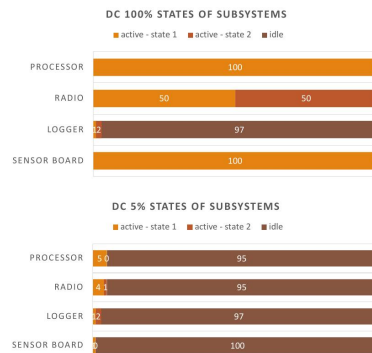
	model 1: 100%DC	model 2: 5%DC	units
Micro Processor			
current (full operation)	100	5	%
current sleep	0	95	%
Radio			
current in receive	50	4	%
current xmit	50	1	%
current sleep	0	95	%
Logger			
write	1	1	%
read	2	2	%
sleep	97	97	%
Sensor Board			
current (full operation)	100	1	%
current sleep	0	99	%

Some
components
work in parallel

Example: two
DC models

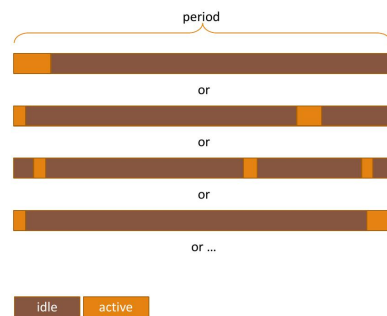
Logger duty
cycle of 3%

Figura 1.8. *Tabella di confronto tra il modello DC 100% e il modello DC 5%.*



Example: two
DC models

Figura 1.9. Rappresentazione grafica degli stati. DC 5% trascorre quasi tutto il tempo idle.

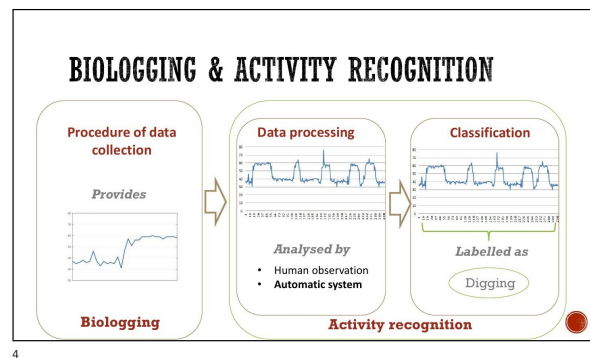


Examples of a
component
with 10% DC

Figura 1.10. Il duty cycle non specifica la posizione dell'attività nel tempo; questo offre flessibilità per la sincronizzazione MAC tra nodi.

1.8.1 Pipeline del Biologging e On-Board Intelligence

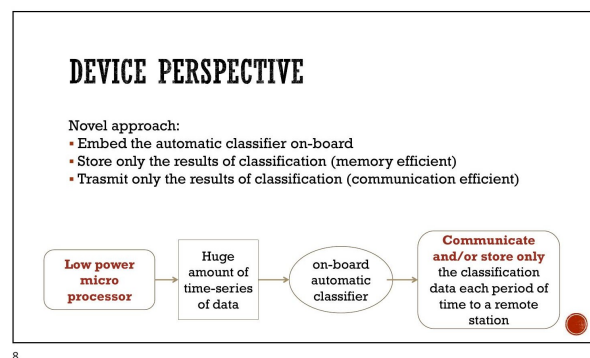
Il monitoraggio con accelerometri e GPS continui esaurirebbe in fretta le batterie. **Approccio innovativo:** spostare l'elaborazione (Machine Learning) **on-board**, analizzare in locale, e trasmettere via radio **solo il risultato semantico**.



4

2

Figura 1.11. Il flusso completo del biologging.



8

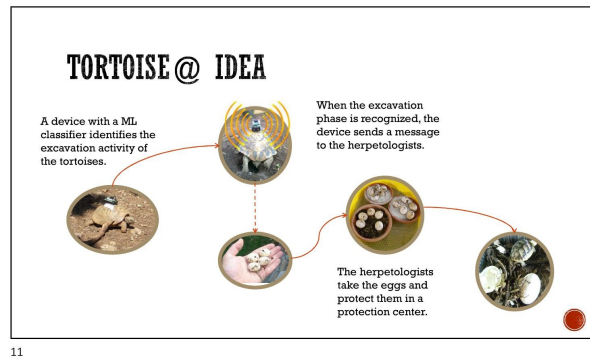
4

Figura 1.12. L'elaborazione e memorizzazione solo dei risultati riduce drasticamente traffico radio e consumi.

1.8.2 Il progetto Tortoise@

Il sistema serve a ritrovare i nidi deposti individuando il segnale accelerometrico dello scavo: - Lo scavo produce un pattern caratteristico imparabile. - Le tartarughe nidificano in giorno in luoghi caldi.

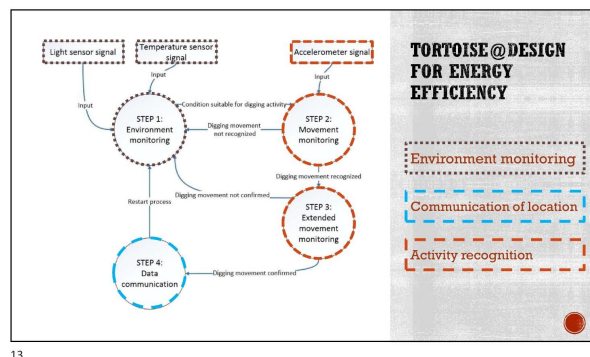
10/05/2021



11

Figura 1.13. Il sistema invia notifiche GPS agli erpetologi appena riconosce il nido.

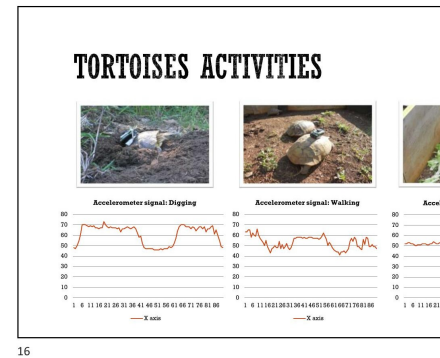
10/05/2021



13

Figura 1.14. Macchina a stati finiti: usa sensori luce/temperatura molto economici come “gate” per azionare i dispendiosi accelerometri solo quando ha senso.

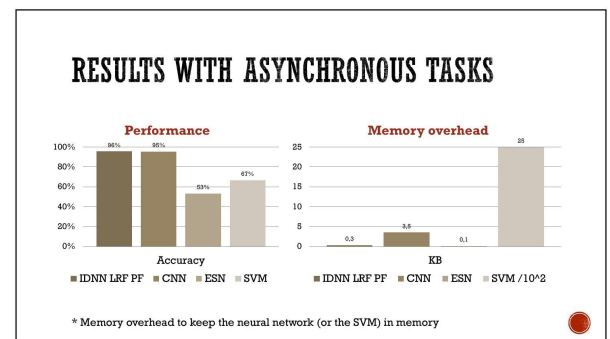
1.8.3 Classificazione



16

Sensori MicaZ a 1Hz su asse X, finestre asincrone e sincrone.

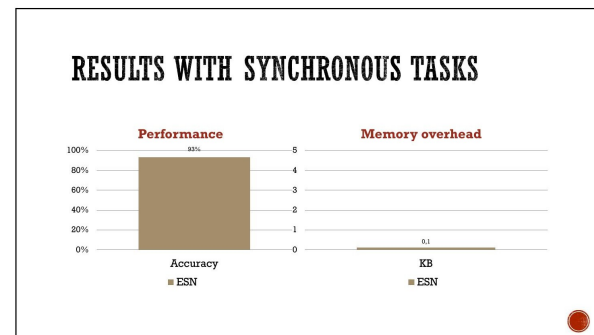
10/05/202



19

Risultati (Task asincrono 300 sec):

- La rete **IDNN** ottiene il 95% di accuratezza richiedendo solo **0,3 KB** di RAM, battendo le CNN che ne chiedono 3,5 KB. Su MCU piccoli è essenziale. SVM richiede decine di KB per i vettori.



20

Risultati (Task sincrono in streaming):

- Con **Echo State Network (ESN)**, accuratezza 93% con appena **0,1 KB** di RAM.

Nota – Sintesi del risparmio

Rispetto al cloud che richiede di trasmettere e/o memorizzare 13 MB, Tortoise@ on-board conserva max 3 KB alla volta e usa il modulo radio solo poche volte in 4 mesi.

1.9 9. Embedded Programming e Arduino

Gli **sistemi embedded** sono progettati per svolgere una funzione dedicata, tramite co-design hardware-software, che integra microprocessori (microcontrollori) operanti a basso livello per fornire il controllo del dispositivo ospite. A causa dei vincoli (spesso senza un OS vero e proprio), si richiedono requisiti stringenti di **Real-Time** (la correttezza temporale è essenziale quanto quella logica) e di alta tolleranza ai fault.

1.9.1 Cross-Compilazione e Modelli di Esecuzione

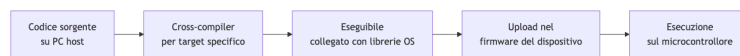


Figura 1.15. Pipeline di sviluppo per sistemi embedded, dal codice sorgente su PC host fino all'esecuzione sul microcontrollore

Poiché scarseggia RAM (es. Arduino Uno ha solo 2 KB), usare multipli thread dotati di stack separati non è sostenibile. Due modelli principali emergono:

1.9.2 Il Modello Arduino

Event loop sincrono a singolo thread. `loop()` viene iterato indefinitamente.

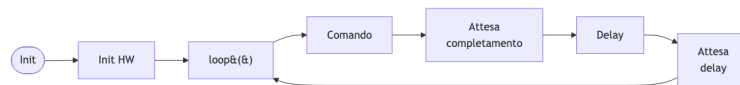


Figura 1.16. Modello di esecuzione di Arduino basato su singolo thread, esecuzione sequenziale e polling sincrono.

1.9.3 Il Modello TinyOS

Progettato per efficienza e modularità asincrona senza thread overhead. Sviluppato in tre entità: - **Comandi**: funzioni down-ward verso l'hardware. - **Eventi**: up-call che astraggono interrupt pre-emptando il flusso. - **Task**: coda sequenziale non-preemptive per il lavoro deferito dagli event handlers.

1.9.4 Arduino e Interrupt Esterni

La piattaforma Arduino Uno basa tutto sul ATmega328.

Oltre al bloccante `delay()`, offre gestione asincrona tramite gli **interrupt esterni** (`attachInterrupt()`) sui pin digitali 2 e 3. Regole vitali per i gestori di interrupt (*handler*): - Devono essere rapidissimi, delegando tutto il lavoro extra. - Dentro l'handler `delay()` o le funzioni basate su `millis()` non funzionano. - Le variabili modificate nell'handler e lette nel loop devono essere marcate come **volatile** in C, imponendo letture forzate in RAM.

1.9.5 Sleep Mode in Arduino

L'ATmega328P supporta vari Sleep mode gestibili tramite *LowPower.h*. Il più utile e profondo per azzerare il DC in inattività è **Power-Down**, che spegne il clock principale consentendo risvegli unicamente tramite interrupt esterni.

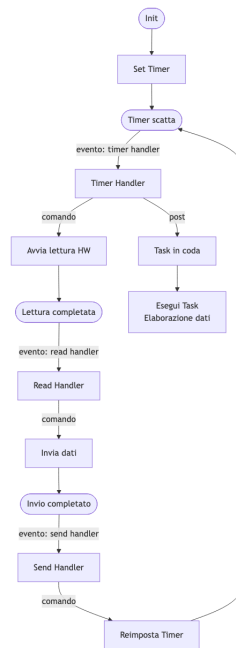


Figura 1.17. Flusso di esecuzione basato su eventi in TinyOS, in cui gli eventi gestiscono immediatamente l'hardware e delegano l'elaborazione ai task asincroni.

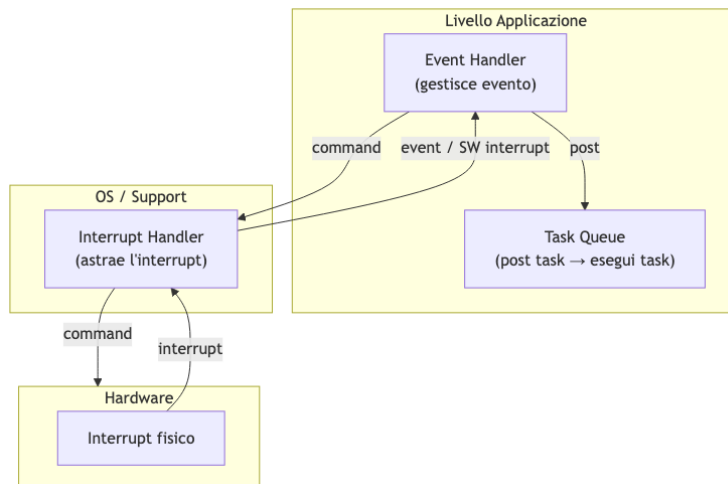


Figura 1.18. Modello a tre livelli per la gestione degli eventi nei sistemi embedded, in cui l'hardware genera interrupt, il sistema operativo li astrae in eventi e l'applicazione li gestisce tramite event handler e task.

ARDUINO UNO

**AVR Arduino microcontroller
Atmega328**

- SRAM 2KB (data)
- EEPROM 1KB
- Flash memory 32 KB (program)

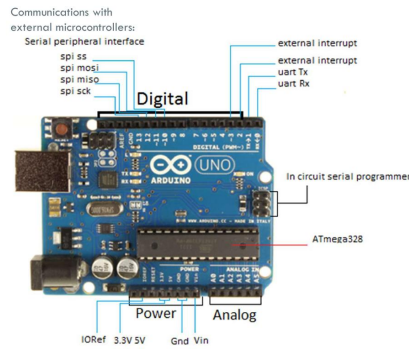


Figura 1.19. Scheda Arduino Uno con pinout annotato

INTERRUPTS — EXAMPLE

Connections:

- A button to digital input 2
- With 10K Ω resistor
- A led to digital output 7
- With 220 Ω resistor

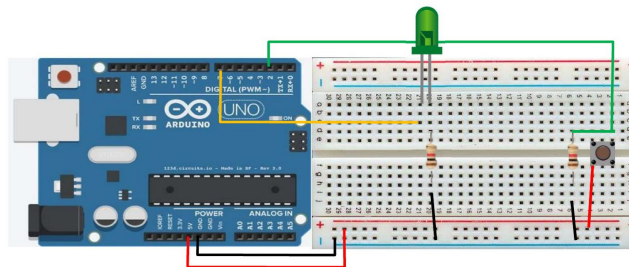


Figura 1.20. Schema circuito Arduino con interrupt

1.10 10. Domande d'esame frequenti

- Cos'è un Cyber-Physical System e in cosa si distingue da un sistema puramente informatico o fisico?
- Descrivere le quattro generazioni di Internet e le caratteristiche che differenziano l'IoT.
- Quali funzionalità offre una piattaforma IoT e perché è necessario uno strato software intermedio?
- Cosa si intende per vendor lock-in e vertical silos?
- Quali sono le tre macro-categorie di scenari d'uso del 5G?
- Descrivere le configurazioni di integrazione Tipo A, B, C e D: quando è necessario un gateway applicativo?
- Quali sono i requisiti di sicurezza (ITU-T Y.2066) e qual è il ruolo del gateway?
- Quali sono le differenze tra Edge, Fog e Cloud Computing nell'IoT?
- Come funziona il Federated Learning e quali sono i principali attacchi a cui è soggetto?
- Perché i database NoSQL sono preferiti ai DB relazionali nei contesti IoT?
- Quali sono le tre interpretazioni della Legge di Moore e come si applicano all'IoT?
- Perché tenere la radio in modalità "listen" è quasi costoso quanto ricevere dati?
- Come si calcola il duty cycle di un componente e qual è il trade-off tra edge processing e offloading in cloud?
- Qual è la differenza tra biotelemetria e bio-logging?
- Come funziona la macchina a stati di Tortoise@ per ottimizzare l'efficienza energetica e confronta IDNN, CNN, ESN e SVM.
- Quali sono le principali differenze tra un sistema embedded e un PC general purpose e in cosa consiste il co-design?
- Confrontare il modello Arduino con il modello TinyOS (eventi, comandi e task).
- Quali regole vanno rispettate nello scrivere un Interrupt Handler (es. `volatile`, blocchi)?
- Descrivere le modalità di sleep dell'ATmega328P.

Capitolo 2

02 - Protocolli Applicativi (MQTT e CoAP)

Questa lezione analizza l'evoluzione delle architetture di rete e degli stack di protocolli pensati per l'Internet of Things, culminando con un'analisi strutturale del protocollo MQTT e del protocollo CoAP. Vengono esplorati i paradigmi di comunicazione, le logiche di connessione, la strutturazione dei topic e i meccanismi di affidabilità.

2.1 I Requisiti dell'IoT e la Conventional Internet Protocol Suite

Perché un oggetto diventi a tutti gli effetti un dispositivo IoT deve essere connesso a Internet. Tradizionalmente, i sistemi si interfacciano alla rete globale tramite la **Internet protocol suite** convenzionale, strutturata sul binomio TCP/IP affiancato da un livello applicativo come HTTP. Questa suite si articola su tre livelli: il **livello di rete** (**IP** per indirizzamento e routing best-effort); il **livello di trasporto** (**TCP** per garanzie di consegna o **UDP** per minor overhead); il **livello applicativo** (**HTTP** con architettura client/server).

Questo stack è stato pensato per dispositivi *resource-rich*. Per i nodi IoT risulta troppo pesante: operano su reti instabili (*lossy*), con hardware a bassissimo consumo energetico (*low power*) e risorse estremamente limitate (*constrained*). Questi vincoli impongono una ridefinizione completa dei requisiti:

Requisiti di Rete	Impatto sul Networking
Scalabilità / Ridondanza	Necessità di reti multi-hop e architetture mesh.
Sicurezza	Deve essere configurabile in base alle capacità dei singoli dispositivi.
Indirizzamento	Richiede uno spazio di indirizzamento scalabile e protocolli a basso overhead.
Requisiti del Dispositivo	Impatto sul Livello Applicativo
Basso consumo / A batteria	Progettazione di applicazioni con basso duty-cycle.
Capacità limitata (memoria/CPU)	Necessità di protocolli con footprint minimo e bassa complessità.
Basso costo	Aumenta la richiesta di affidabilità, introducendo ulteriori vincoli fisici.

2.2 Introduzione al Protocollo MQTT

Definizione – MQTT

MQTT (*Message Queuing Telemetry Transport*) è un protocollo di messaggistica leggero di tipo publish/subscribe, progettato per ambienti con risorse limitate e connettività instabile. Standardizzato da OASIS (versione di riferimento 3.1.1).

Ideato nel 1999, la sua leggerezza si manifesta in tre modi: code footprint ridotto, basso consumo di banda e overhead di pacchetto minimo. MQTT si appoggia su TCP/IP operando sulla **porta 1883** in chiaro e sulla **porta 8883** in **SSL/TLS** (che introduce un overhead computazionale significativo per le MCU).

Il protocollo concentra la complessità sul lato server (il broker), mantenendo l'implementazione client semplice. È *data agnostic* (non si cura del payload) e implementa meccanismi di garanzia della consegna (QoS).

2.3 Il Paradigma Publish / Subscribe

Tip – Pub/Sub e i tre disaccoppiamenti

Il paradigma introduce tre forme di *decoupling* ideali per l'IoT: - **Space decoupling**: publisher e subscriber non si conoscono (non condividono IP/porta). - **Time decoupling**: non devono essere operativi contemporaneamente. - **Synchronization decoupling**: le operazioni sui client non sono bloccanti durante publish o receive.

Il paradigma pub/sub è *loosely coupled*. I **publisher** e i **subscriber** agiscono come client. Il **broker** riceve tutti i messaggi, li filtra e li distribuisce. Le operazioni cardine sono quattro: *Publish*, *Subscribe*, *Notify* e *Unsubscribe*.

La scalabilità è superiore rispetto all'architettura client/server. Le modalità di filtraggio sono tre: **topic-based** (MQTT), **content-based** (il broker ispeziona il payload, ma impedisce la cifratura), e **type-based** (filtraggio semantico, richiede forte integrazione col linguaggio). Publisher e subscriber devono accordarsi *a priori* sui topic, e il publisher non può assumere che ci sia qualcuno in ascolto.

2.4 Il Modello MQTT: Connessione e Flusso Operativo

L'infrastruttura opera ai livelli 5-6 (ISO/OSI), sopra TCP. I dispositivi devono conoscere hostname e porta del broker in anticipo.

2.4.1 L'Instaurazione della Connessione

Il client invia il pacchetto **CONNECT**: - **Client ID** (obbligatorio): univoco. Se vuoto, richiede *Clean Session* a **true**. - **Clean Session** (opzionale): **false** richiede una *persistent session* (salva stato, iscrizioni e QoS 1). - **Username/Password** (opzionali): per autenticazione (in chiaro senza TLS). - **Will flags** (opzionali): testamento per disconnessioni anomale. - **KeepAlive** (opzionale): timeout (in secondi) entro cui inviare almeno un pacchetto (ping).

Il broker risponde con **CONNACK** (Connect Acknowledgement), contenente i flag *Connection Accepted/Refused* e *Session Present*.

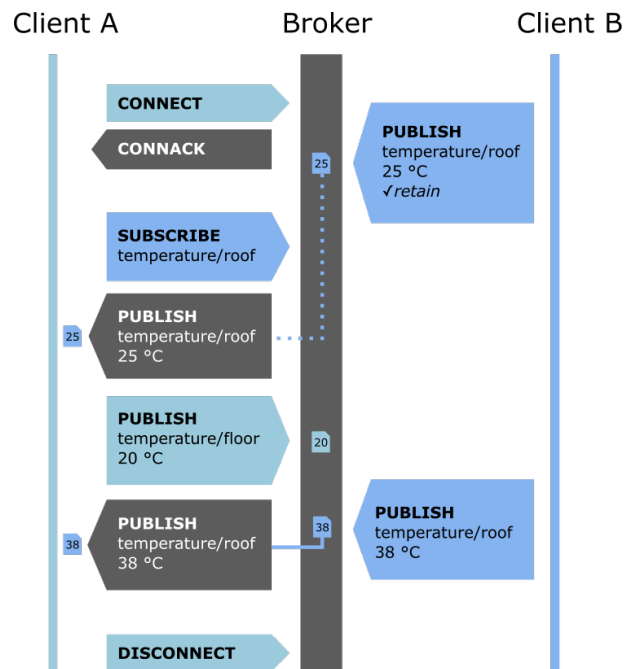


Figura 2.1. Fonte: Wikimedia Commons — Il flusso completo: Client A si connette, Client B si sottoscrive, Client A pubblica.

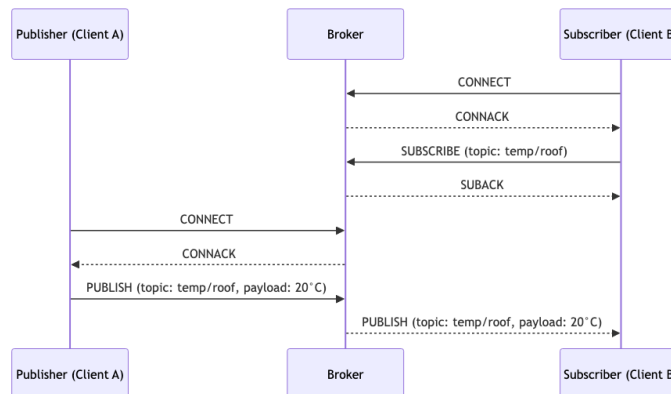


Figura 2.2. Questo schema illustra il flusso base di interazione tra publisher, broker e subscriber in MQTT, mostrando la connessione, sottoscrizione e pubblicazione dei dati.

2.4.2 Sottoscrizione e Ricezione

Un client invia **SUBSCRIBE** contenente un `packetId` e l'elenco dei topic con il QoS richiesto. Il broker risponde con **SUBACK** (ritorna 128 per fallimento, o 0, 1, 2 per successo col *maximum QoS granted*). Per revocare si usa **UNSUBSCRIBE** (risposta **UNSUBACK**).

2.4.3 Gestione e Best Practices dei Topic

Gerarchia separata dal carattere /. I subscriber possono usare le **wildcard**: - **+**: sostituisce un singolo livello (es. `home/+/presence`). - **#**: sostituisce tutti i livelli successivi (es. `home/#`). I topic che iniziano con **\$** sono riservati al broker per statistiche (es. `$SYS/broker/uptime`).

Attenzione – Best practices per i topic

- Non iniziare il topic con / (es. `/home`).
- Non usare spazi; mantenere le stringhe corte; usare ASCII/UTF-8.
- Includere il `clientId` (es. `sensor1/temperature`) per limitare i permessi.
- Usare topic specifici anziché aggregati.
- Evitare l'abuso del wildcard `#`.

2.4.4 Pubblicazione

Messaggio **PUBLISH**: - **topicName**: stringa del topic di destinazione. - **payload**: dati (binari, testo, JSON). - **packetId**: 16 bit, usato per QoS > 0. - **retainFlag**: se attivo, il broker conserva l'ultimo messaggio per i nuovi subscriber. - **dupFlag**: duplicato per ritrasmissioni. - **qos**: livello di Quality of Service.

2.5 I Meccanismi della Quality of Service (QoS)

Il QoS regola la garanzia di consegna. È importante notare che il QoS tra publisher e broker è indipendente da quello tra broker e subscriber. I messaggi sono tracciati da un `packetId` a 16 bit.

Nota – I tre livelli QoS a confronto

Livello	Nome	Garanzia	Meccanismo	Duplicati
QoS 0	At most once	Nessuna	Nessun ACK, no mem.	No
QoS 1	At least once	Almeno una	PUBACK	Possibili
QoS 2	Exactly once	Esattamente una	Handshake a 4 fasi	No

- **QoS 0**: Modalità *best effort*. Ottima per dati che invecchiano rapidamente.
- **QoS 1**: Il broker memorizza il messaggio e risponde con **PUBACK**. Se l'ACK non arriva, si ritrasmette con `dupFlag` attivo (possibili duplicati).
- **QoS 2**: Elimina duplicati e garantisce singola consegna. Comporta un handshake a quattro fasi: **PUBLISH** → **PUBREC** → **PUBREL** → **PUBCOMP**.

2.6 Meccanismi di Affidabilità Avanzati

2.6.1 Sessioni Persistenti

Se *Clean Session* è **false**, il broker salva per il `clientId` le sottoscrizioni e i messaggi non consegnati (QoS 1/2). Anche il **client** deve salvare i messaggi non-acked. Non vanno usate se si pubblica solo a QoS 0 o se perdere messaggi vecchi è tollerabile.

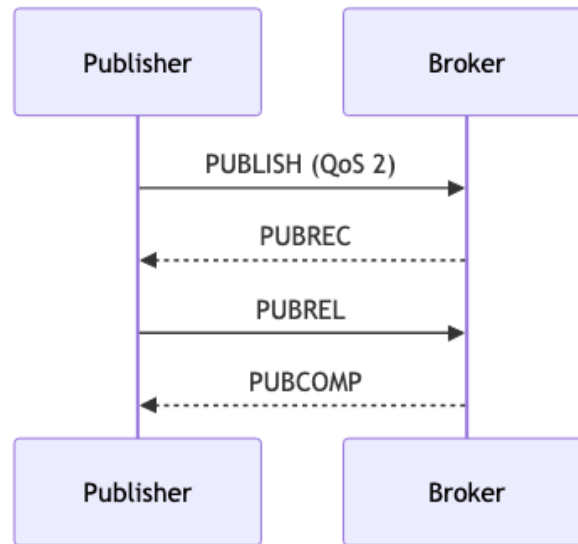


Figura 2.3. Flusso di messaggi nel QoS 2 per garantire una singola consegna esatta tra client e broker, evitando perdita di dati e duplicati.

2.6.2 Messaggi Trattenuti (*Retained Messages*)

Messaggi inviati con `retainFlag = true`. Il broker ne mantiene uno per topic e lo invia istantaneamente a ogni nuovo iscritto (utile per trasmettere lo stato attuale senza aspettare nuovi aggiornamenti, es. dispositivo “ON”). Indipendenti dalle sessioni persistenti.

2.6.3 Last Will & Testament (Testamento)

Configurato durante la connessione (CONNECT) con i parametri `lastWillTopic`, `lastWillQoS`, `lastWillMessage` e `lastWillRetain`. Se il client si disconnette *anormalmente*, il broker pubblica il messaggio. Avviene per 4 cause: errore di I/O, PINGREQ mancato (keep alive), chiusura TCP brusca, errore protocollo lato broker. (Se la disconnessione è tramite DISCONNECT, viene scartato).

2.6.4 Keep Alive

Mitiga il problema di TCP di non accorgersi dei peer offline. Il client invia **PINGREQ** entro l'intervallo specificato in CONNECT, e il broker risponde con **PINGRESP**.

2.7 Struttura dei Pacchetti MQTT

Ogni **MQTT Control Packet** comprende: 1. **Fixed Header** (min 2 byte): - **Byte 1**: Tipo pacchetto (4 bit) + Flag (4 bit). Es. nel PUBLISH i flag indicano DUP, QoS e Retain. Negli altri spesso è 0,0,0,0 (eccetto PUBREL/SUBSCRIBE/UNSUBSCRIBE che usano 0,0,1,0). - **Byte 2+**: *Remaining Length* (lunghezza variabile). 2. **Variable Header**: Presente solo in alcuni pacchetti (es. `packet identifier` di 2 byte per QoS > 0). 3. **Payload**: Es. il `client identifier` in CONNECT. È assente nei pacchetti di controllo (PINGREQ, DISCONNECT, ACK QoS).

I tipi principali: CONNECT(1), CONNACK(2), PUBLISH(3), PUBACK-PUBREC-PUBREL-PUBCOMP(4-7), SUBSCRIBE(8), SUBACK(9), ecc.

2.8 Implementazione Pratica: MQTT su Arduino

La libreria **PubSubClient** è la più diffusa: limitata volontariamente per leggerezza (niente SSL/TLS, no QoS 2, max 128 byte payload). - Si istanzia con broker IP, porta e un **callback** (funzione eseguita event-driven all'arrivo dei messaggi). - API base: `connect(...)`, `publish(...)`, `subscribe(...)`, `disconnect()`. - L'esecuzione ciclica di `loop()` è essenziale per mandare i PINGREQ (mantenere attiva la connessione) e processare la callback.

2.9 MQTT vs HTTP e il Problema della Scalabilità

HTTP impone connessioni client/server simmetriche e rigide (comunicazioni 1-a-1). Con molti dispositivi, **HTTP non scala bene**. MQTT scala sul broker, che gestisce il *fan-out* senza gravare sui client, scambiando pacchetti compatti (byte vs documenti testuali voluminosi di HTTP).

Attenzione – Limiti strutturali di MQTT

1. Il broker è un **single point of failure**. 2. L'overhead del broker può crescere eccessivamente su reti enormi. 3. L'uso di **TCP** richiede risorse maggiori, causa *wake-up time* alti e maggior consumo di batteria rispetto a protocolli UDP.

2.10 CoAP: L'Alternativa per Reti Vincolate

Per dispositivi con bassissime risorse (RAM/ROM) e reti lossy (es. 6LoWPAN a 10 kbit/s), **CoAP** (*Constrained Application Protocol*, RFC 7252) rimpiazza MQTT.

Abbraccia un paradigma **client/server**, dove i sensori operano da *server* e i software centrali da *client*. Usa un'architettura **REST** (GET, PUT, POST, DELETE) per accedere alle risorse esposte.

Nota – Caratteristiche Tecniche di CoAP

- Utilizza **UDP/IP** al posto di TCP, annullando l'overhead di instaurazione connessione. - Formati compatti e *resource directory* integrata per la discovery. - Sicurezza nativa robusta (livelli RSA 3072 bit). - Supporta anche un modello asincrono parziale.

Tip – Quando scegliere CoAP vs MQTT

- Scegli **MQTT** per massimo disaccoppiamento (spazio/tempo) e in presenza di un broker affidabile. - Scegli **CoAP** per reti ultra-vincolate (UDP), per evitare il broker centrale (peer-to-peer) e per sicurezza crittografica *by default*.

Domanda – Possibili domande d'esame

- Limiti dello stack TCP/IP per l'IoT e requisiti imposti. - Paradigma publish/subscribe, i tre disaccoppiamenti e differenze col client/server. - Ruolo del broker, filtraggio messaggi e gestione topic. - Parametri CONNECT (Clean Session, Last Will, KeepAlive). - Wildcard MQTT (+ e #) e best practices dei topic. - Differenze e funzionamento dei livelli QoS (0, 1, 2) e l'handshake a 4 fasi. - Differenza tra sessione persistente e retained message. - I quattro scenari di innesco del Last Will & Testament. - Struttura dei pacchetti (Fixed Header, Variable, Payload). - Limiti della libreria PubSubClient per Arduino (es. assenza QoS 2). - Architettura e limiti di MQTT vs HTTP (single point of failure, TCP vs UDP). - CoAP: differenze con MQTT, paradigma REST, l'uso di UDP e in quali scenari preferirlo.

Capitolo 3

Livello MAC e Reti WSN

3.1 1. Protocolli MAC per Reti Wireless Multi-hop (Lezione 17)

3.1.1 Il problema dell'efficienza energetica

Nei sistemi IoT e nelle reti di sensori wireless, il protocollo MAC non ha come unico obiettivo l'arbitrazione dell'accesso al mezzo: deve minimizzare il consumo energetico. Il radio transceiver è il componente più energivoro; la strategia fondamentale è ridurre il **duty cycle** (la frazione di tempo in cui rimane attivo).

Tip – Intuizione chiave

Un nodo con la radio sempre accesa scarica la batteria rapidamente. Un nodo con la radio sempre spenta risparmia energia ma non può comunicare. Il protocollo MAC deve trovare il compromesso tra energia, latenza e banda.

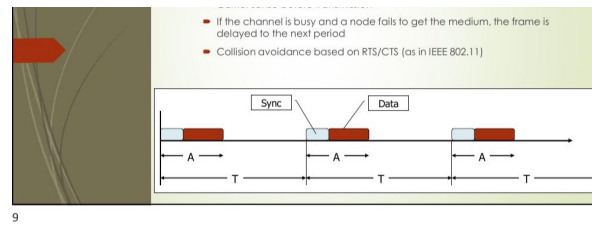
Approcci per ridurre il consumo:

Approccio	Descrizione	Esempi
Sincronizzazione dei nodi	I nodi si svegliano contemporaneamente	S-MAC, IEEE 802.15.4
Preamble sampling	Preambolo lungo, campionamento periodico	B-MAC, X-MAC, BoX-MAC
Polling	Master coordina slave tramite beacon	Bluetooth, IEEE 802.15.4

3.1.2 Sincronizzazione: S-MAC

S-MAC (*Sensor MAC*) è progettato per reti wireless multi-hop. L'idea è che i nodi vicini si accordino su un periodo di ascolto condiviso per poi tornare in sleep. - **Sincronizzazione locale:** Ogni nodo trasmette frame SYNC in broadcast per annunciare la propria schedule. Nodi adiacenti adottano schedule comuni creando "isole di sincronizzazione". - **Comunicazione:** Un nodo trasmette durante il periodo di ascolto del ricevitore. Esegue il carrier sense (RTS/CTS) prima di trasmettere. S-MAC usa l'*adaptive duty cycle*: se un nodo sente un RTS/CTS, mantiene la radio accesa per velocizzare i salti multi-hop.

Latenza multi-hop e Clock Drift: La latenza si accumula (proporzionale a $n \times T_{listen_period}$) se le schedule nei salti successivi non sono allineate. Per contrastare il clock drift (deriva hardware), i frame SYNC periodici mantengono i nodi allineati nel tempo.



9

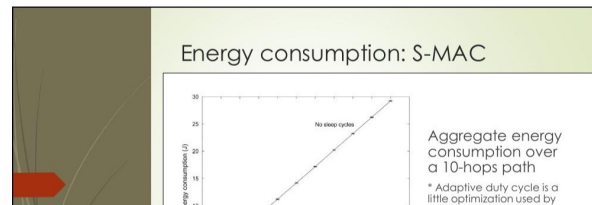
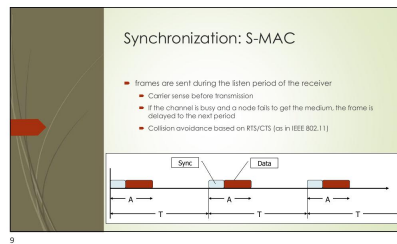
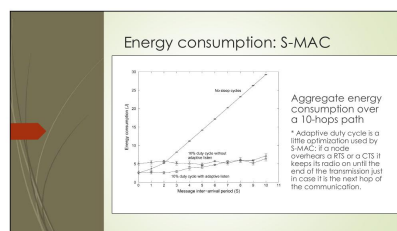


Figura 3.1. Il frame Sync e Data trasmessi nel periodo di attività.



9



10

5

Figura 3.2. Consumo energetico totale in funzione del traffico.

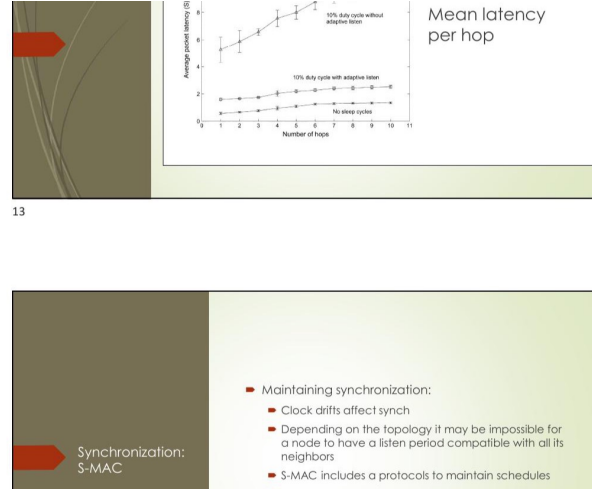


Figura 3.3. Latenza media per hop in S-MAC

3.1.3 Preamble Sampling: B-MAC

B-MAC (*Berkeley MAC*) elimina la necessità di coordinazione temporale sfruttando il *Low-Power Listening* (LPL). Il ricevitore attiva la radio periodicamente per brevissimi istanti per campionare il canale. Il mittente, quando deve trasmettere, invia un preambolo molto lungo (la cui durata deve essere superiore al periodo di sleep del ricevitore) seguito dal frame dati.

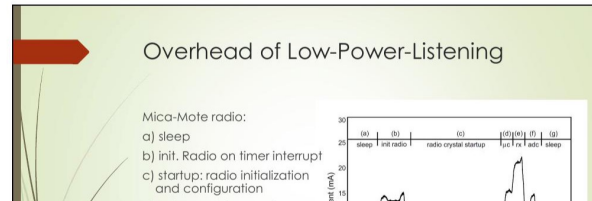
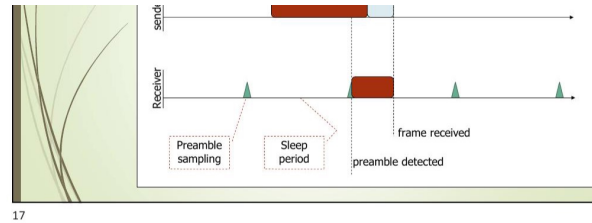


Figura 3.4. Schema temporale del preamble sampling B-MAC

Modello energetico: Siano: - f_d = frequenza di trasmissione dei dati (Hz) - f_c = frequenza di campionamento del preambolo - p_t, p_r, p_i = potenza in trasmissione, ricezione, idle.

Per il **trasmittente**, il duty cycle è:

$$DC_{data} = f_d \cdot (t_{preamble} + t_{frame})$$

$$DC_{check} = f_d \cdot t_{check}$$

L'energia consumata in t secondi:

$$E_T(t) = t[p_t \cdot DC_{data} + p_r \cdot DC_{check} + p_i \cdot (1 - DC_{data} - DC_{check})]$$

Per il **ricevitore**:

$$DC_{data} = f_c \cdot (t_{listen} + t_{data})$$

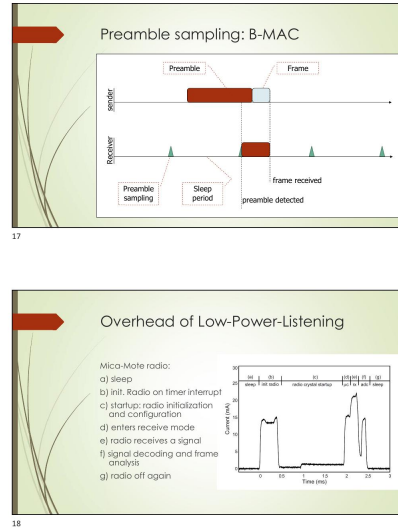


Figura 3.5. Profilo di corrente del radio Mica-Mote durante il ciclo LPL

$$DC_{check} = f_c \cdot t_{check}$$

$$E_R(t) = t[p_r \cdot DC_{data} + p_r \cdot DC_{check} + p_i \cdot (1 - DC_{data} - DC_{check})]$$

Il preambolo lungo costa molta energia in trasmissione, ma garantisce grandissimi risparmi per tutti i ricevitori in ascolto, che necessitano di campionamenti piccolissimi. B-MAC ha un solo parametro di setup: l'intervallo di check.

3.1.4 Evoluzioni: X-MAC e BoX-MAC

X-MAC: Ottimizza il costo in trasmissione dividendo il lungo preambolo in brevi frame ripetuti contenenti l'ID del destinatario. Quando il ricevitore si sveglia e legge il proprio ID, interrompe immediatamente il trasmettitore inviando un ACK precoce, spingendolo a inviare il frame effettivo e risparmiando energia.

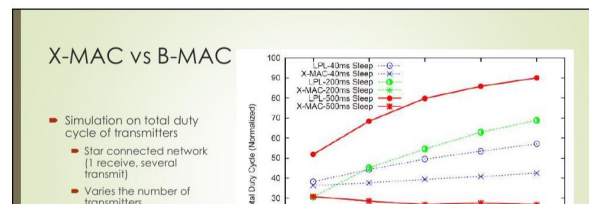
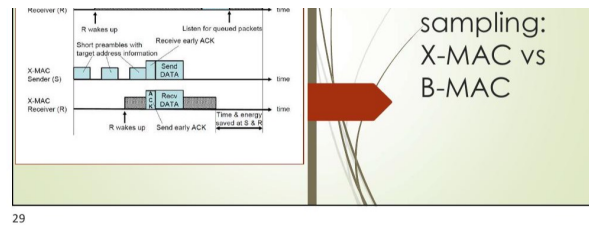


Figura 3.6. Confronto temporale X-MAC vs B-MAC

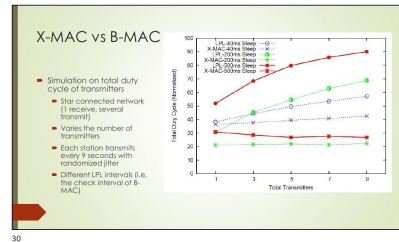
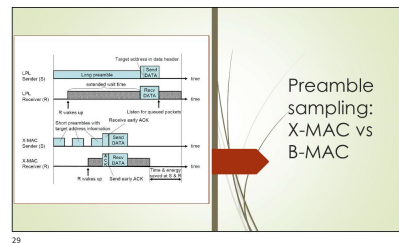


Figura 3.7. Duty cycle totale trasmettenti: X-MAC vs B-MAC al variare del numero di nodi

BoX-MAC: Un'ulteriore evoluzione in cui il “preambolo” diventa il frame dati stesso, ripetuto in loop. Il ricevitore intercetta un frammento, aspetta l'inizio della replica successiva, la riceve per intero e manda l'ACK per fermare il mittente. È eccellente per reti di sensori che scambiano frame molto leggeri.

3.1.5 Polling

Tipico di Bluetooth e **IEEE 802.15.4** (combinato con la sincronizzazione). L'architettura è asimmetrica: un nodo master annuncia messaggi pendenti tramite *beacon*, i nodi slave possono tenere la radio spenta a piacimento e si svegliano per prelevare il messaggio dal master inviando una richiesta esplicita.

3.2 2. Lo Standard IEEE 802.15.4 (Lezione 18)

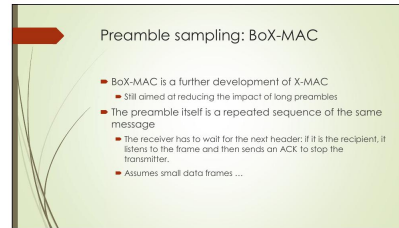
Fornisce le specifiche del **Physical layer** e del **MAC layer** per reti **LR-WPAN** (Low-Rate Wireless Personal Area Network). Opera con dispositivi a risorse limitate, garantendo consumi ridotti, coesistenza con Wi-Fi (802.11) e Bluetooth (802.15.1).

3.2.1 Physical Layer

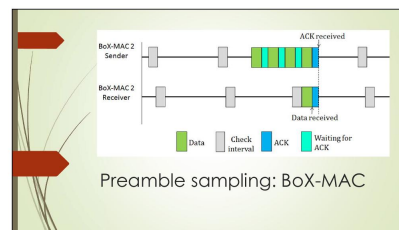
Opera in bande libere da licenza: | Banda | Regione | Data rate | Canali | |—|—|—|—| | 868–868.6 MHz | Europa | 20 kbps | 1 (canale 0) | | 902–928 MHz | Nord America | 40 kbps | 10 (canali 1–10) | | 2400–2483.5 MHz | Mondiale | 250 kbps | 16 (canali 11–26) |

Eroga *Data Service* per ricezione/trasmissione e *Management Service* per funzioni fisiche: - **Energy Detection (ED)**: Misura l'energia sul canale (su una finestra di 8 simboli) con soglia di 10 dB sopra la sensibilità radio. - **Link Quality Indicator (LQI)**: Stima della qualità del frame ricevuto per alimentare le logiche di routing. - **Clear Channel Assessment (CCA)**: Verifica se il canale è libero tramite 3 modalità (tramite ED, tramite Carrier Sense, o combinazione logica).

Struttura della PPDU (PHY Protocol Data Unit): - *SHR*: Synchronization Header (preambolo e SFD). - *PHR*: PHY Header (lunghezza, max 127 byte). - *PHY Payload*: PSDU.



31

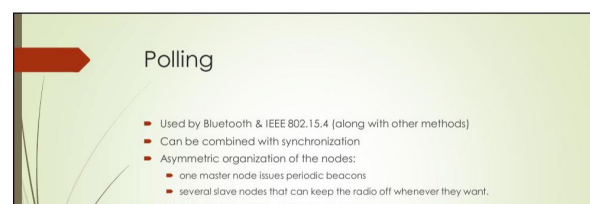


32

16

Figura 3.8. *Diagramma temporale BoX-MAC*

33

Figura 3.9. *Routing multi-hop in BoX-MAC*

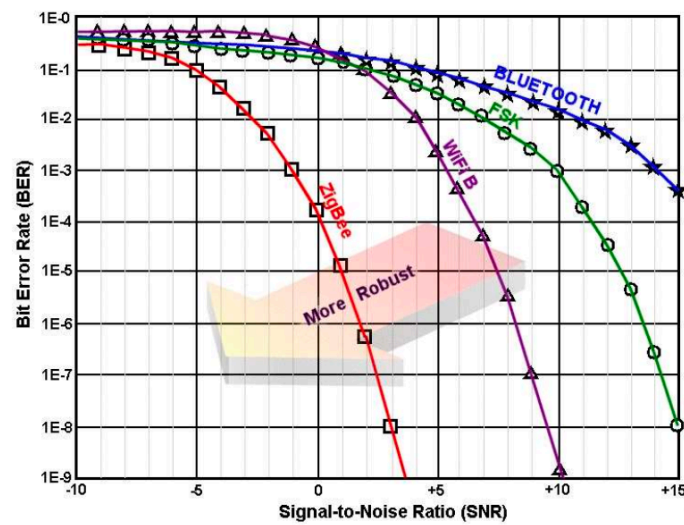


Figura 3.10. L'ottima resistenza di 802.15.4 al rumore (BER vs SNR).

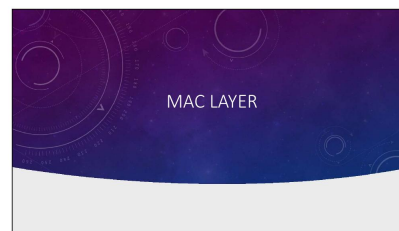
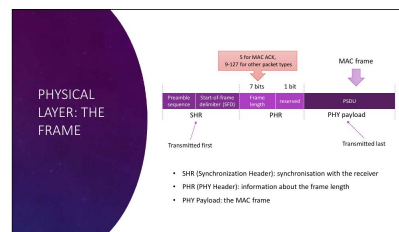
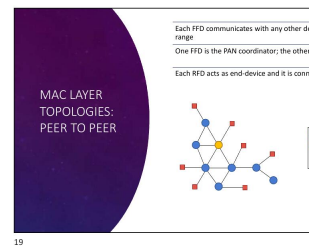
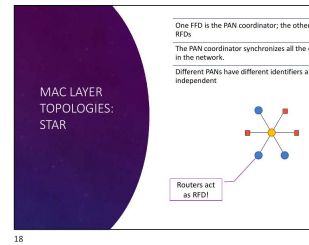


Figura 3.11. Struttura della PPDU IEEE 802.15.4

3.2.2 MAC Layer: Dispositivi e Topologie

Esistono due classi di nodi: - **RFD (Reduced Function Device)**: Logica minima, destinato agli end-device (sensori). Può associarsi a un solo nodo padre FFD. - **FFD (Full Function Device)**: Logica MAC completa. Può agire da Router, da *PAN Coordinator* (che crea la rete) o da end-device generico.

Le topologie supportate sono **Stella** (Coordinator al centro, comunicazioni dirette solo verso di lui) e **Peer-to-**



Peer (FFD connessi in mesh/albero, RFD appesi come foglie).

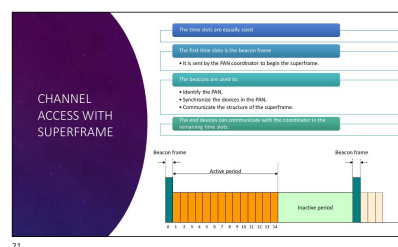
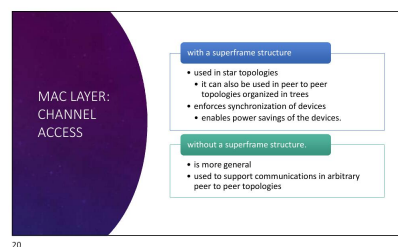


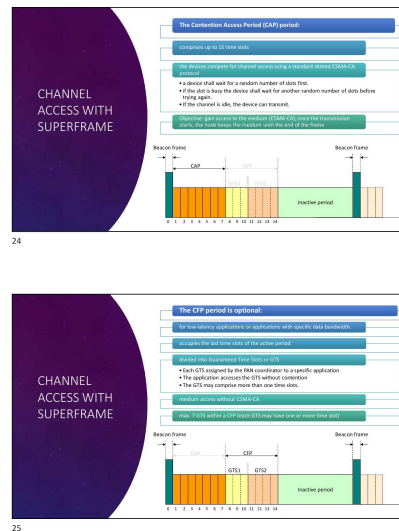
Figura 3.12. Topologia Peer-to-Peer IEEE 802.15.4

3.2.3 Accesso al Canale e Superframe

Modalità Beacon-Enabled: Il coordinatore trasmette beacon periodici definendo un **Superframe**. Il superframe è diviso in periodo attivo (in cui si comunica) e inattivo (tutti in sleep). Il periodo attivo contiene fino a 16 slot suddivisi in: - **CAP (Contention Access Period)**: accesso tramite *slotted CSMA-CA*. - **CFP (Contention Free Period)**: opzionale, slot garantiti **GTS** assegnati a dispositivi specifici per determinismo temporale.

Attenzione – Il CAP non può essere eliminato

Il CAP è indispensabile per gestire messaggi asincroni (associazioni, richieste GTS) e deve esistere anche se è presente il CFP.



12

Figura 3.13. Struttura del superframe CAP e CFP

Modalità Non Beacon-Enabled: Nessun superframe. Usa **unslotted CSMA-CA**. I coordinatori devono restare sempre accesi. L'end-device preleva dati tramite polling al coordinatore.

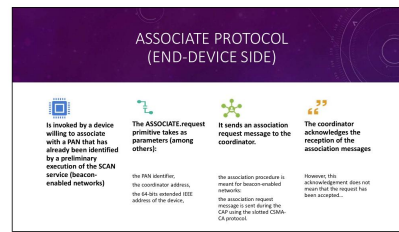
3.2.4 Associazione e Sicurezza

Il MAC fornisce una suite di primitive: DATA, ASSOCIATE, SCAN, GTS, POLL. L'associazione si basa su: Scansione (SCAN), richiesta dell'end-device al PAN Coordinator (ASSOCIATE.request), accettazione del coordinatore con assegnazione di short address a 16-bit (ASSOCIATE.response via polling indiretto).

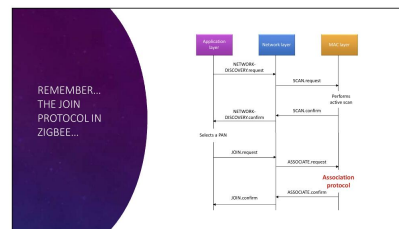
A livello MAC è integrato il supporto base per la sicurezza (chiavi simmetriche fornite dal network layer): controllo accessi, cifratura dati (link o group key), integrità e "Sequential Freshness" per evitare attacchi di replay.

3.3 3. Lo Standard ZigBee: Network Layer (Lezioni 9)

ZigBee è il protocollo delle reti IoT concepito dalla *ZigBee Alliance* su basi IEEE 802.15.4, offrendo self-healing, basso costo, lunga durata batterie e scalabilità. Costruisce sopra PHY e MAC i propri **Network (NWK)** e **Application Layer**.



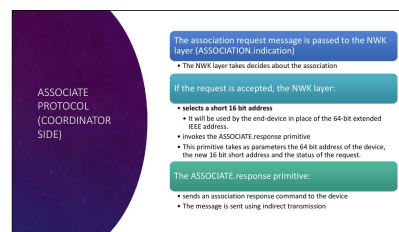
44



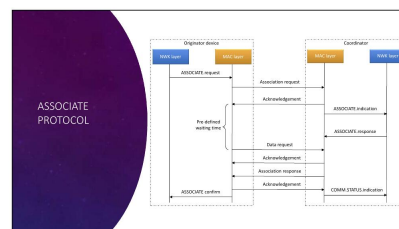
45

21

Figura 3.14. Diagramma flusso MAC Data Service e protocollo di associazione ZigBee



48



49

23

Figura 3.15. Diagramma di sequenza del protocollo ASSOCIATE IEEE 802.15.4

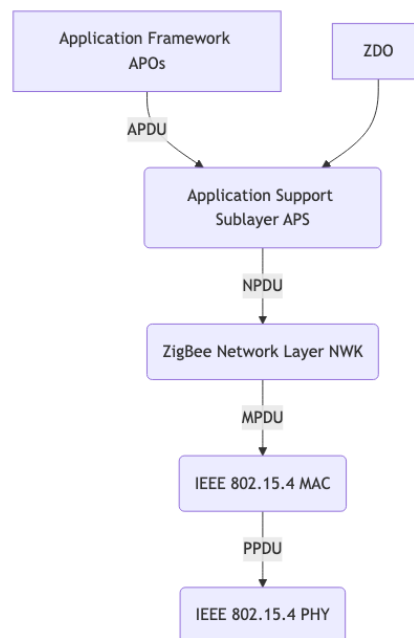
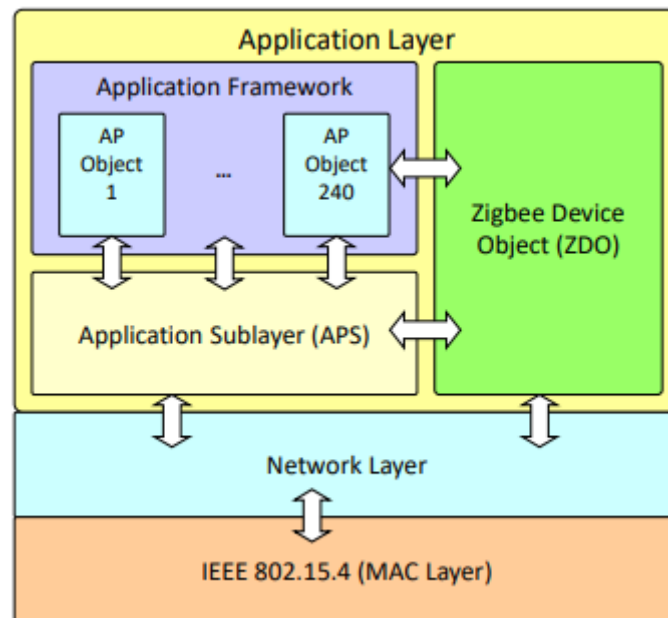
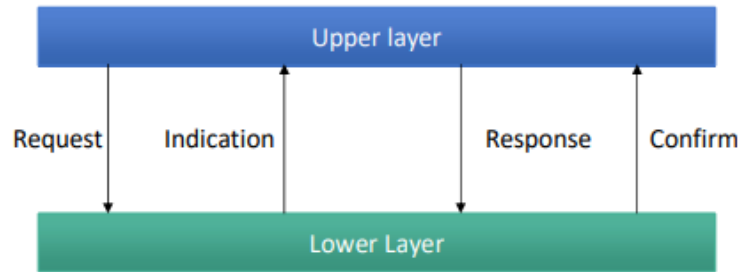


Figura 3.16. Stack protocollare di ZigBee e incapsulamento da Application ad IEEE 802.15.4.



3.3.1 Formazione della Rete e Indirizzamento ad Albero

La rete ha tre ruoli logici: **Network Coordinator** (FFD, crea la rete), **Router** (FFD, instrada traffico), **End-Device** (RFD/FFD). La creazione della PAN (*Network Formation*) è eseguita dal Coordinator con un energy/active scan per trovare canali liberi. Sceglie un PAN ID, si autoassegna l'indirizzo 0×0000 e avvia i beacon.

L'ingresso in rete avviene tramite *Join through Association* (device avvia scansione e si associa al padre) oppure *Direct Join* (un nodo forza l'aggregazione al figlio conosciuto).

Allocazione degli Indirizzi (Tree Structure): ZigBee pre-distribuisce blocchi di indirizzi per assegnazioni veloci e scalabili, evitando collisioni. I parametri hardware del coordinator sono fissi: R_m (max router figli), D_m (max end-device figli), L_m (profondità massima albero). La grandezza del blocco per un router a profondità d è $C_{skip}(d)$:

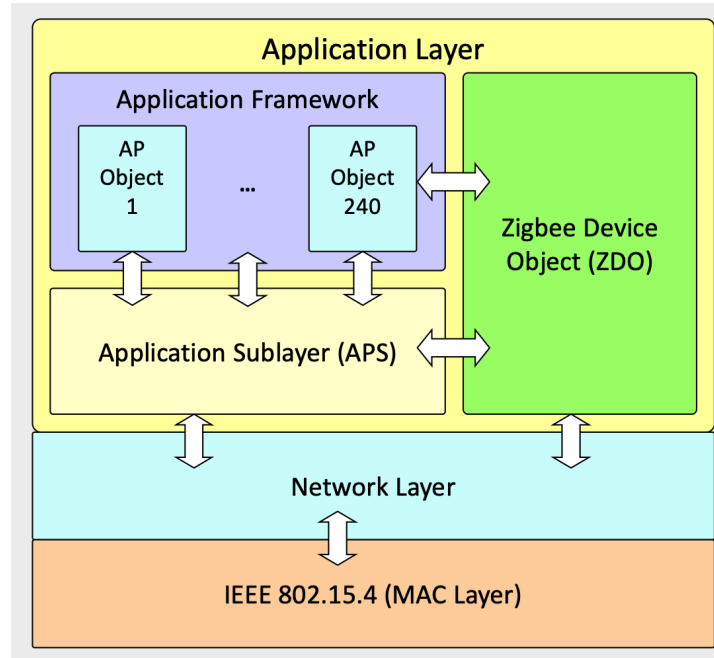
$$C_{skip}(d) = \begin{cases} 1 & \text{se } d = L_m \\ 1 + R_m \cdot C_{skip}(d+1) + D_m & \text{altrimenti} \end{cases}$$

I figli router riceveranno gli indirizzi saltando segmenti pari a $C_{skip}(d+1)$.

3.3.2 Routing

ZigBee supporta instradamento differenziato, e le logiche possono coesistere: - **Tree Routing:** segue i rami logici dell'albero formati in fase di join. È compatibile col beaconing ma può usare cammini non ottimali. - **Mesh Routing:** ispirato ad AODV. Usa la **Routing Table** per inoltri diretti. In caso di rotta mancante, usa il *Route Discovery Protocol* diffondendo in broadcast messaggi **RREQ** per trovare la destinazione. Il **RREP** torna indietro memorizzando il "Residual Cost" nelle routing table. Molto flessibile e robusto, ma non permette una rete globalmente sincronizzata a superframe.

3.4 4. Application Layer: ZDO, APS e ZCL (Lezione 12)



Il livello Applicativo poggia su tre pilastri: 1. **Application Framework ed Endpoints**: Fino a 240 applicazioni coesistenti (APO) su un singolo device. Ogni applicazione risiede in un endpoint numerato da 1 a 240. L'Endpoint 0 è sempre fisso e riservato al componente ZDO. 2. **ZigBee Device Object (ZDO)**: È il manager che controlla localmente la logica della rete: offre i servizi di *Device e Service Discovery* all'utente, gestisce le direttive di *Binding* e assicura il *Node Management* secondo il proprio ruolo hardware. 3. **Application Support Sublayer (APS)**: È il "Transport layer" di ZigBee. Filtra indirizzi, assicura la ricezione tramite conferme applicative end-to-end, e memorizza le tabelle multicast (Group Management).

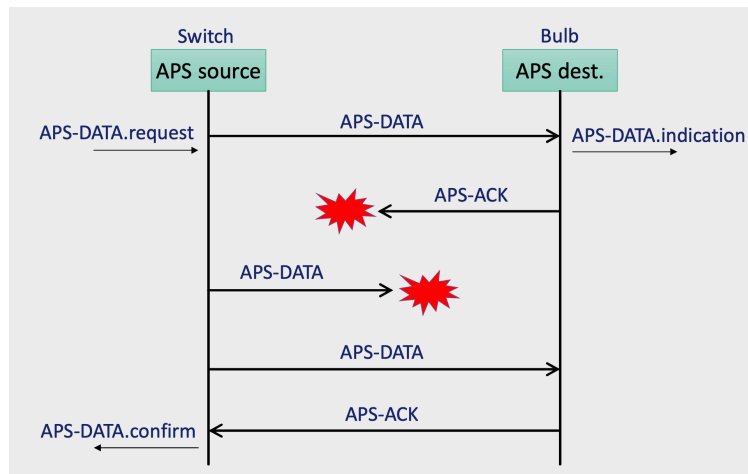
3.4.1 La ZigBee Cluster Library (ZCL) e i Profili

Un **Cluster** è l'unità base funzionale: definisce formalmente i **Comandi** attuabili e gli **Attributi** di stato esposti. È identificato da 16-bit (es. OnOff Cluster 0 × 0006). Un **Application Profile** raggruppa comportamenti per interoperabilità (es. Home Automation 0104). *Device IDs* identificano visivamente il dispositivo (una lampadina o termostato) ma NON servono per il Service Discovery.

La **ZCL** è un archivio enorme di cluster pre-confezionati dalla ZigBee Alliance per evitare che le industrie reinventino la logica applicativa. Supporta un'architettura **Client/Server** in cui il *Dynamic Attribute Reporting* spinge in automatico le variazioni al client risparmiando l'energia di un continuo polling in lettura.

3.4.2 APS Binding e Indirizzamento Indiretto

Il **Binding** (servito dal livello APS) crea una connessione virtuale persistente tra un cluster su un nodo e uno remoto, pur non conoscendone le coordinate logiche a priori. Dispositivi basici si limitano a inviare dati verso un endpoint generico. L'APS layer sfrutta la **Binding Table** combinata con l'**Address Map Table** (che fissa la traduzione tra indirizzo fisico MAC e quello volatile NWK 16-bit) per instradare a livello software il pacchetto. Anche in caso di riavvio logico della rete, il ponte applicativo non si spezza mai.



Esempio Applicativo (Indoor Localization): Tramite l'analisi dell'RSSI radio è supportato il *Remote Positioning* (le ancore recepiscono e computano i segnali mobili) o il *Self Positioning* (il mobile triangola i fari delle ancore). In ZigBee, tramite l'*RSSI Location Cluster*, paradossalmente il nodo mobile si atteggia a "Server" del Cluster, in quanto sorgente dei dati logici, e il nodo ancorato legge come "Client".

3.5 5. ZigBee Security (Lezione 13)

ZigBee definisce un framework imperativo basato su: *key establishment*, *key transport*, *frame protection* e *device management*. La logica fondamentale prevede che **il livello che origina il messaggio debba essere colui che provvede alla messa in sicurezza**. Quasi tutto il traffico subisce cifratura a livello NWK per proteggere dal "theft of service" (tranne i fisiologici messaggi aperti per l'associazione iniziale). Il framework impone severe routine di gestione per le perdite di sincronizzazione crittografiche o overflow.

3.5.1 Architettura di Sicurezza

- **NWK Layer:** Si assicura che ogni hop sia cifrato e integro e inserisce contatori anti-replay (*Freshness*).
- **APS Layer:** Eroga i servizi crittografici veri e propri, cifra carichi di lavoro end-to-end e trasporta in via logica le chiavi.
- **ZDO:** Il manager strategico che dirige le policy e ordina all'APS le manovre da fare.

Chiavi a 128-bit: 1. **Network key:** Usata su base collettiva. 2. **Link keys:** Sessioni dirette tra un nodo A e un nodo B (per comunicazioni esclusive). 3. **Master key:** Non è usata per cifrare traffico dati vivo, ma viene sfruttata da algoritmi hash unidirezionali per generare in autonomia nuove Link Keys.

3.5.2 Symmetric-Key Key Establishment (SKKE) e Trust Center

SKKE è il processo gestito dal livello APS che permette a due nodi di ricavare una nuova Link Key per derivazione tramite una sequenza *challenge-response* di scambio di numeri effimeri aleatori e hash applicati a un pre-segreto (Trust Provisioning). I nodi derivano la chiave, poi confermano per scrupolo di aver ottenuto lo stesso match.

In ogni rete blindata c'è e deve esserci **un solo Trust Center** designato, autorizzato a validare l'ingresso nella rete e ad erogare centralmente il materiale crittografico ai nodi al momento del *join* (sfruttando chiavi Master per aprire prima tunnel provvisori se necessario). Nelle reti domestiche, le funzioni del Trust Center sono semplicemente incluse e assorbite dal Coordinator.

Nota – Domande d'esame e Concetti Chiave di Riepilogo

- Compromesso MAC: energia vs. latenza e throughput. Differenza tra Sync (S-MAC), Preamble Sampling (B-MAC, X-MAC, BoX-MAC) e Polling. - B-MAC: necessità di un preambolo più lungo della finestra di sleep. X-MAC riduce il tempo interrompendolo al read dell'ID; BoX-MAC ripete in catena il pacchetto stesso. - IEEE 802.15.4: Bande libere (es. 2.4 GHz globale). Ruoli (FFD, RFD). Formato superframe: l'indispensabilità del CAP a prescindere dal CFP/GTS. Protocollo di Associazione MAC.

- Architettura ZigBee: I tre livelli superiori (NWK, ZDO, APS, Application Framework). $C_{skip}(d)$ nell'indirizzamento ad albero statico. Mesh Routing (RREQ/RREP stile AODV) contro Tree Routing (supporta beaconing ma rotta fissa). - ZDO (Endpoint 0), ZCL e Profiling. Il binding indiretto tramite APS. Il concetto di Client/Server nel Reporting dinamico. - Sicurezza NWK e Trust Center univoco in rete per distribuzione e validazione delle chiavi crittografiche (Network, Link e Master Key). SKKE challenge-response.

Capitolo 4

Gestione Energetica: Energy Harvesting e Neutralità

Il problema di fondo dei dispositivi IoT è energetico: alimentarli con una batteria di capacità finita costringe a scegliere tra prestazioni e durata. Una batteria grande permette lunga vita, ma aumenta dimensioni, peso e costo; un processore a basso consumo estende la vita ma limita potenza di calcolo e portata radio. L'**energy harvesting** (raccolta di energia) rappresenta un'alternativa strutturale: invece di ottimizzare il consumo, si raccoglie energia dall'ambiente e si elimina — o si riduce drasticamente — il vincolo della batteria finita.

Se la sorgente è abbondante e disponibile in modo continuo o periodico, il dispositivo può funzionare «per sempre». Questa prospettiva sposta l'obiettivo del progettista dalla massimizzazione della vita utile alla massimizzazione delle prestazioni a parità di disponibilità energetica.

4.1 Definizioni fondamentali

Tre concetti strutturano l'intero dominio:

- **Energy source** (sorgente di energia): la fonte primaria (sole, vento, vibrazioni, ecc.) da cui l'energia viene estratta.
- **Harvesting source** (tecnologia di raccolta): il componente fisico — cella solare, turbina, cristallo piezoelettrico, antenna — che converte la sorgente in energia elettrica. La produzione varia nel tempo e in funzione delle condizioni ambientali, ed è generalmente al di fuori del controllo del progettista.
- **Load** (carico): l'energia consumata dal dispositivo per le proprie attività. Un dispositivo IoT ha molteplici sottosistemi (processore, radio, storage, trasduttori/ADC), ciascuno con stati propri e consumi differenti; il carico varia quindi nel tempo a seconda delle attività in esecuzione.

Definizione – Harvesting System

Un sistema che supporta un carico variabile alimentato da una sorgente di harvesting variabile, anche quando la potenza istantanea della sorgente non corrisponde al carico.

Il problema chiave è il **disaccoppiamento** tra produzione e consumo: la sorgente produce quando l'ambiente lo consente, il dispositivo consuma quando deve eseguire i propri task.

Esistono due approcci principali per adeguare produzione e consumo: 1. **Adattare il carico** alla disponibilità energetica — ad esempio riducendo la frequenza di campionamento. 2. **Usare un buffer energetico** — una batteria ricaricabile o un supercapacitore che accumula l'eccesso e lo cede nei momenti di deficit.

In pratica nessuno dei due approcci è sufficiente da solo: il carico non può essere ridotto arbitrariamente senza compromettere la funzionalità, e il buffer è fisicamente limitato e non ideale (ha perdite ed efficienza di carica < 1).

4.2 Architetture di harvesting

4.2.1 Harvest-Use

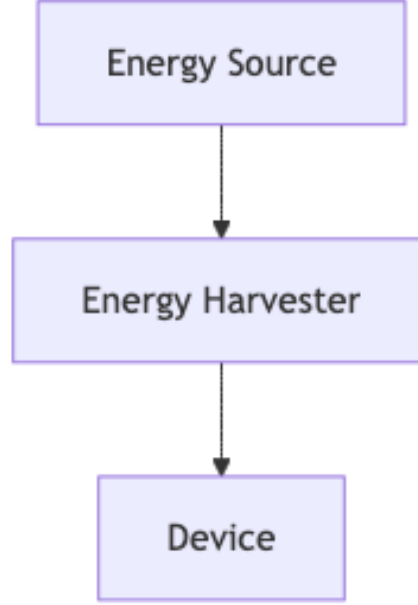


Figura 4.1. Schema logico dell'architettura Harvest-Use in cui l'energia raccolta alimenta direttamente il dispositivo

Fig. — Architettura Harvest-Use: la sorgente alimenta direttamente il dispositivo senza buffer.

Nell'architettura **Harvest-Use** l'energia è raccolta esattamente quando serve. Non esiste un buffer: la potenza prodotta $P_s(t)$ viene consumata istantaneamente. Il dispositivo è operativo solo se:

$$P_s(t) \geq P_c(t)$$

Questo comporta due forme di spreco: - Quando $P_s(t) < P_c(t)$: il dispositivo si spegne per mancanza di energia. - Quando $P_s(t) > P_c(t)$: l'eccesso $P_s(t) - P_c(t)$ va perduto.

4.2.2 Harvest-Store-Use

Fig. — Architettura Harvest-Store-Use: l'energia in eccesso viene accumulata nel buffer e prelevata nei momenti di deficit.

Nell'architettura **Harvest-Store-Use** l'energia raccolta viene immagazzinata nel buffer ogni volta che è disponibile e prelevata quando la produzione è insufficiente o i task del dispositivo richiedono più energia. Il buffer disaccoppia temporalmente produzione e consumo.

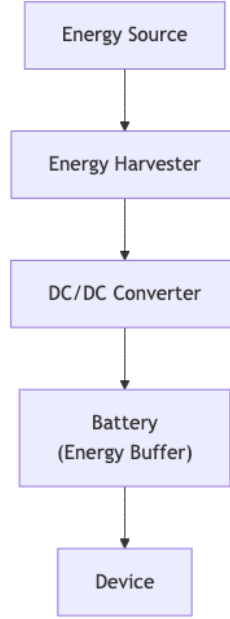


Figura 4.2. Schema dell'architettura Harvest-Store-Use che include un convertitore e un buffer di energia (batteria)

Buffer ideale e non ideale

Un buffer ideale ha capacità infinita, nessuna perdita ed efficienza di carica $\eta = 1$. Il dispositivo è sempre operativo se la carica prodotta più quella iniziale eccede il consumo per ogni intervallo:

$$\int_0^T P_c(t) dt \leq \int_0^T P_s(t) dt + B_0 \quad \forall T \in (0, \infty)$$

Un buffer reale ha capacità massima $B_{\max}$, efficienza $\eta < 1$ e potenza di leakage $P_{\text{leak}}(t)$. La carica B_T al tempo T deve soddisfare: **Conservazione dell'energia (necessaria e sufficiente):**

$$B_T = B_0 + \eta \int_0^T [P_s - P_c]^+ dt - \int_0^T [P_c - P_s]^+ dt - \int_0^T P_{\text{leak}} dt \geq 0$$

Capacità finita (sufficiente, non necessaria):

$$B_{\max} \geq B_0 + \eta \int_0^T [P_s - P_c]^+ dt - \int_0^T [P_c - P_s]^+ dt - \int_0^T P_{\text{leak}} dt$$

Tip – Esercizio sul buffer non ideale

Dispositivo con $\eta = 95\%$, $B_0 = 400 \text{ mAh}$, leakage trascurabile. Intervallo $[0, 4s]$: $P_s = 80 \text{ mA}$, $P_c = 150 \text{ mA}$. Consumo supera produzione. Produzione = 0.089 mAh , Consumo = 0.167 mAh . $B_4 = 400 + 0.089 - 0.167 = 399.922 \text{ mAh}$ Intervallo $[4s, 10s]$: $P_s = 80 \text{ mA}$, $P_c = 20 \text{ mA}$. Produzione supera consumo. Produzione = 0.133 mAh , Consumo = 0.033 mAh . $B_{10} = 399.922 + 0.95(0.133 - 0.033) = 400.017 \text{ mAh}$

4.3 Fonti di energia e classificazione

Le sorgenti si classificano su due assi: **controllabilità** e **prevedibilità**.

Controllabilità	Descrizione	Esempi
Completamente controllabile	L'energia è disponibile su richiesta	Torcia shake-to-power, sorgenti RF dedicate
Parzialmente controllabile	Influenzabile ma non deterministica	Tag RFID in ambiente RF non uniforme
Non controllabile	Raccolta solo quando disponibile	Sole, vento, termica ambientale

Le sorgenti non controllabili si classificano in prevedibili (sole) e non prevedibili (vibrazioni sismiche). La prevedibilità è fondamentale per la pianificazione energetica.

4.3.1 Dettaglio delle Tecnologie

- **RF harvesting:** I tag RFID passivi si alimentano esclusivamente con l'energia RF del reader.
- **Piezoelettrico:** Converte deformazione meccanica in differenza di potenziale (es. solette per scarpe, pulsanti wireless).
- **Energia solare ed Eolica:** Il sole produce in modo dipendente da latitudine, meteo e stagioni, con forte variabilità. La combinazione sole-vento (vento notturno, sole diurno) aumenta la copertura temporale, sebbene un harvesting non garantisca l'operatività continua per casi peggiori se mal dimensionati.

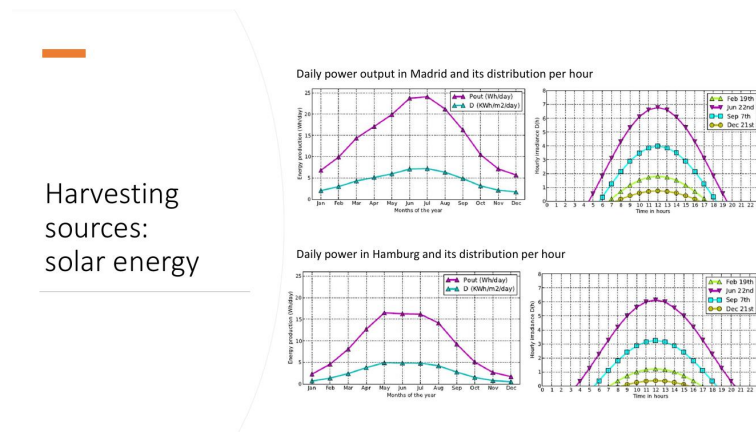


Figura 4.3. Produzione solare giornaliera a Madrid e Amburgo con distribuzione oraria

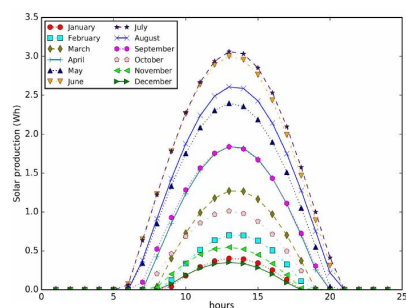
4.3.2 Tecnologie di storage

- **Batterie Li-ion:** Preferite per l'IoT grazie a efficienza altissima (99.9%), bassissimo self-discharge e alta densità.
- **Supercapacitori:** Ricariche infinite, efficienza elevata. Ideali con energia disponibile a intervalli o forte jitter, tenuti carichi con trickle charge.

4.4 Misurare la carica e la produzione

Tutte le tecniche di power management richiedono info fresche sulla carica della batteria e la produzione.

La tensione a circuito aperto di una batteria decresce in modo circa **lineare** nel range operativo.



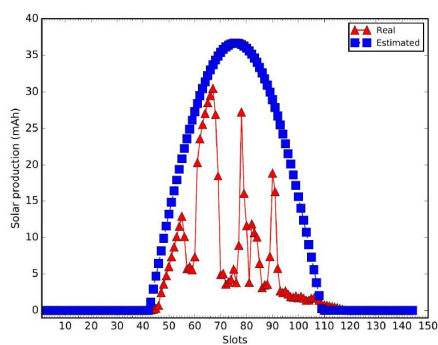
Ideal energy production

- Solar panel KL-SUN3W
- Expected energy production in Madrid
- Time frame of 24 hours, from 0:00 to 23:59

Considerations:

- availability of harvested energy varies very much in different periods of the year ... (e.g. December vs June)
- Usual solution: oversize the harvesting subsystem and the battery to match with the worst conditions

Figura 4.4. *Produzione ideale mensile del pannello KL-SUN3W a Madrid*



Real vs estimated production

- Solar energy production per 10-minutes slot in a day:
 - Measurements taken on the 27th of March 2017 in Ciudad Real (Spain)
 - Production in mAh

... cloudy day...

Figura 4.5. *Produzione solare reale vs stimata — Ciudad Real, 27 marzo 2017*

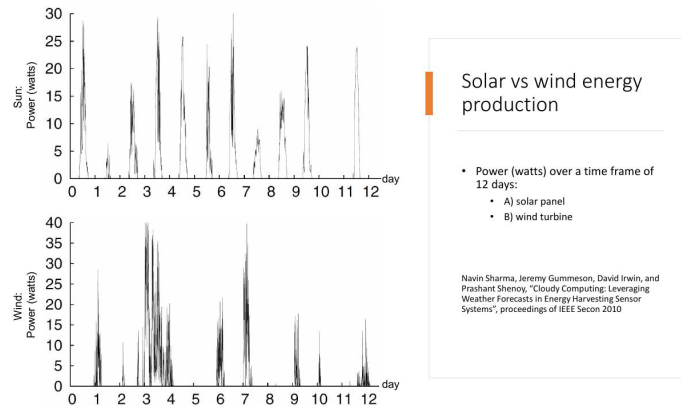


Figura 4.6. Confronto produzione solare vs eolica su 12 giorni

measuring the battery level

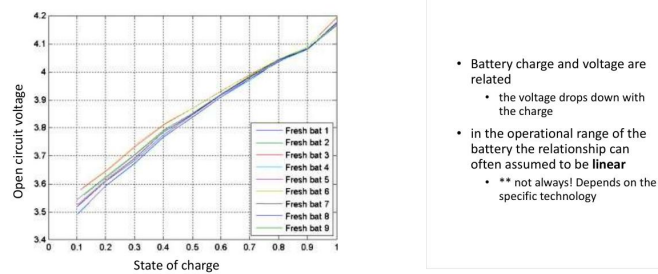


Figura 4.7. Tensione a circuito aperto vs stato di carica per batterie Li-ion

Tramite un ADC a d bit si legge la tensione per risalire alla carica:

$$x_{\max} = 2^d - 1 \quad x_{\min} = \text{ROUND}\left(\frac{v_{\min}}{v_{\text{ref}}}(2^d - 1)\right)$$

$$x = \text{ROUND}\left(\frac{v}{v_{\text{ref}}}(2^d - 1)\right)$$

$$B = B_{\min} + \frac{B_{\max} - B_{\min}}{x_{\max} - x_{\min}}(x - x_{\min})$$

Tip – Esercizio — Stima della carica da output ADC

ADC a 10 bit, max carica 2000 mAh a 10V (batteria piena). A 8V residuo 200 mAh (min). Con $d = 10$ bit si ha $2^d - 1 = 1023$ e $v_{\text{ref}} = 10$ V. $x_{\max} = 1023$, $x_{\min} = 818$. Se ADC è 920: $B = 200 + \frac{920-818}{1023-818} \cdot 1800 \approx 1096$ mAh. Se ADC è 830: $B = 200 + \frac{830-818}{1023-818} \cdot 1800 \approx 305$ mAh. Se ADC è 1023: $B = 2000$ mAh.

4.5 Neutralità energetica e Approccio di Kansal

Definizione – Dispositivo energy-neutral

Un dispositivo è **energy neutral** se riesce a mantenere il livello di prestazione desiderato indefinitamente, senza mai esaurire la batteria.

L'obiettivo è triplice: rimanere energy neutral, evitare lo spegnimento e **massimizzare la utility**.

Kansal affronta il problema per sorgenti **non controllabili ma prevedibili** (es. solare), modulando dinamicamente il duty cycle del dispositivo tramite pianificazione slottata.

4.5.1 Modello Formale: le Due Condizioni

Condizione 1 — Produzione di Energia: L'energia totale $E_T = \int_T P_s(t) dt$ è bounded da un corridoio lineare:

$$\rho_s \cdot T - \sigma \leq E_T \leq \rho_s \cdot T + \sigma$$

Dove ρ_s è il trend medio e σ il burst bound.

Condizione 2 — Consumo (Load): Il consumo $L_T = \int_T P_c(t) dt$ è bounded dall'alto:

$$0 \leq L_T \leq \rho_c \cdot T + \delta$$

Teorema – Teorema di Kansal (Condizione sufficiente)

Se le due condizioni valgono, con efficienza di carica η e leakage ρ_{leak} , il sistema è energy neutral se: 1. $\eta\rho_s \geq \rho_c + \rho_{\text{leak}}$ (trend di produzione copre consumo e perdita) 2. $B_0 \geq \eta\sigma + \delta$ (carica iniziale sufficiente ai burst) 3. $B_{\max} \geq B_0$ (ammissibilità fisica)

4.5.2 Utility e Duty Cycle

Il sistema modula il **duty cycle (dc)** per limitare il consumo e massimizzare una funzione **utility** $u(dc)$:

$$u(dc) = \begin{cases} 0 & \text{se } dc < dc_{\min} \\ \alpha \cdot dc + \beta & \text{se } dc_{\min} \leq dc \leq dc_{\max} \\ u_M & \text{se } dc > dc_{\max} \end{cases}$$

Il consumo di potenza si modella come lineare $p(dc) = \rho \cdot dc + \sigma$.

Condition 1: energy produ

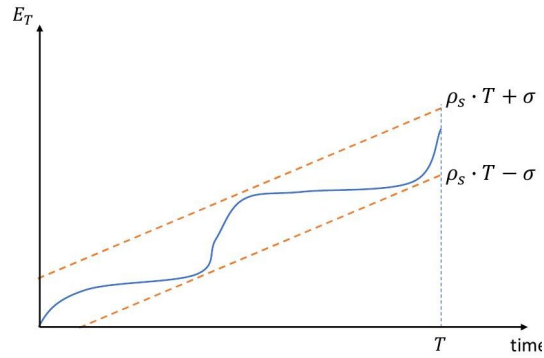


Figura 4.8. *Grafico Condizione 1*

Condition 2: load

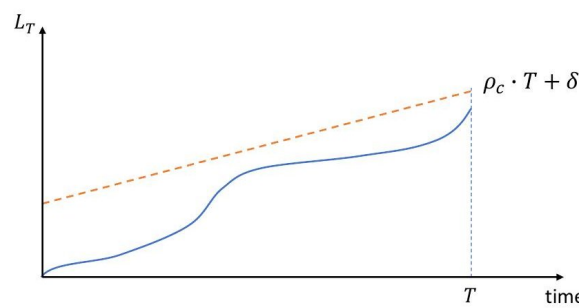


Figura 4.9. *Grafico Condizione 2*

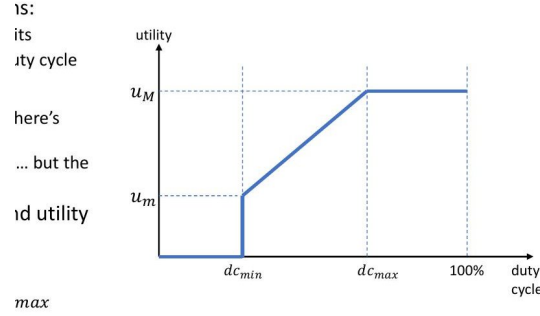


Figura 4.10. Funzione utility vs duty cycle

4.5.3 Previsione: EWMA

Si utilizza il filtro **EWMA** per stimare la produzione slot per slot:

$$\tilde{p}_{j+1}(i) = \alpha \cdot p_j(i) + (1 - \alpha) \cdot \tilde{p}_j(i)$$

La produzione di oggi aiuta a stimare quella di domani. L'errore è più alto attorno a mezzogiorno, ma in generale la produzione solare è ripetibile giorno per giorno.

4.5.4 L'Algoritmo di Kansal

Assegnazione iniziale: per gli slot “di sole” (S , in cui produzione $\geq p_{max}$) il dc è dc_{max} . Negli slot “bui” (D , produzione $< p_{max}$) il dc è dc_{min} . Poi si calcola il **surplus residuo** $p_{res} = B(k+1) - B(1)$.

1. **Caso 1 — Sovraproduzione** ($p_{res} > 0$): Usa l'energia avanzata per alzare al massimo il dc dei dark slot, uno per volta. Il residuo finale alza parzialmente l'ultimo dark slot toccato.
2. **Caso 2 — Sottoproduzione** ($p_{res} < 0$): Manca energia. Controlla che una riduzione uniforme sui sun slot ($|p_{res}|/|S|$) basti (entro $p_{max} - p_{min}$). Se sì, abbassa uniformemente il duty cycle in tutti i sun slot.

Tip – Propriet

È ottimale, semplice, leggero. Se la produzione reale differisce, lo scheduler riadatta la pianificazione dinamicamente per i restanti slot. Limite: si modella solo il dc, ma non scelte tra trasduttori, protocolli, algoritmi.

Esercizio Risolto 1 (Surplus)

Dati: $dc_{max} = 90\%$, $p_{max} = 5$, $u = 100$; $dc_{min} = 50\%$, $p_{min} = 1$, $u = 10$. Formule: $p(dc) = 0.1 \cdot dc - 4$, $u(dc) = 2.25 \cdot dc - 102.5$. Slot $\tilde{p}_s = [0, 0, 0, 2, 6, 11, 10, 7, 3, 0, 0, 0]$. Slot di sole (S): 5, 6, 7, 8 (qui partono a $p_{max} = 5$). Gli altri partono a $p_{min} = 1$. Prodotto = 39. Consumato = $8 \times 1 + 4 \times 5 = 28$. $p_{res} = 11 > 0$. $p_{diff} = 4$. Con 11 surplus, posso portare $11/4 = 2$ dark slot (1 e 2) al massimo. Restano 3 di surplus, usati per lo slot 3, portandolo a un consumo di $1 + 3 = 4$. Per lo slot 3, il $dc = (4 + 4)/0.1 = 80\%$, $u = 77.5$.

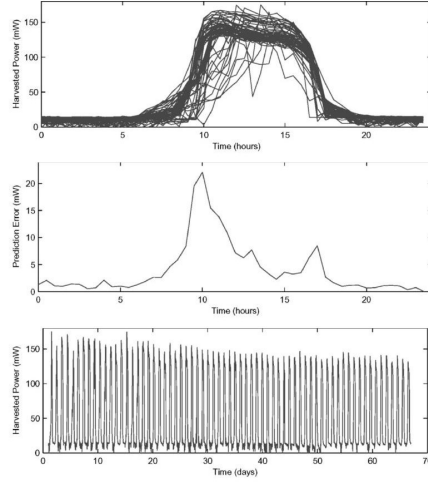


Figura 4.11. Esperimenti con Heliomote e EWMA

Esercizio Risolto 2 (Deficit)

Stessi dati base, ma $p_{min} = 2$. Nuove formule: $\rho = 0.075, \sigma = -1.75$. Slot $\tilde{p}_s = [0, 0, 1, 3, 5, 8, 4, 3, 2, 0, 0]$. Slot di sole: 5, 6. Gli altri a $p_{min} = 2$. Prodotto = 26. Consumato = $10 \times 2 + 2 \times 5 = 30$. $p_{res} = -4 < 0$. Controllo ammissibilità: $(5 - 2) > 4/2 \rightarrow 3 > 2$. Ammissibile. Abbasso consumo dei sun slot di $|p_{res}|/|S| = 4/2 = 2$. Consumo dei sun slot diventa $5 - 2 = 3$. dc dei sun slot: $dc = (3 + 1.75)/0.075 = 63.3\%$. L'utility è 42.4.

4.6 Modello Task-Based per la Neutralità Energetica

Kansal non modella la complessità di vere applicazioni IoT che hanno diverse fasi (Sensing $\rightarrow$ Storing $\rightarrow$ Processing $\rightarrow$ Transmitting) ciascuna con diverse opzioni (trasduttori vari, invio grezzo o elaborato, frequenze). Si passa quindi al concetto di **Task**, cioè una combinazione di opzioni, con costo energetico c_j e utility u_j .

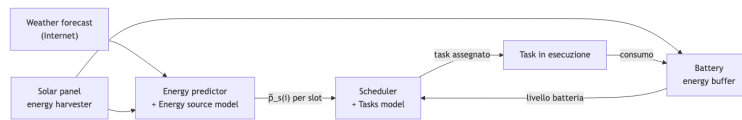


Figura 4.12. Architettura del sistema task-based che mostra il flusso delle previsioni, la schedulazione e il consumo della batteria

Fig. — Architettura del sistema task-based.

4.6.1 Formulazione e Programmazione Dinamica

Lo scheduler assegna **esattamente un task per slot** ($x_{i,j} = 1$), massimizzando l'utility totale $\sum \sum x_{i,j} u_j$ sotto i vincoli che la batteria non si esaurisca mai e alla fine della giornata il livello non sia inferiore a quello iniziale ($B(k+1) \geq B(1)$).

Il problema è **NP-Hard** ma si risolve in modo esatto in tempo **pseudo-polinomiale** usando la programmazione dinamica, esplorando a ritroso (backward) gli slot i e i livelli di batteria discretizzati b :

$$opt(i, b) = \max_{j=1, \dots, n} \{u_j + opt(i+1, B^j(i+1)) : B^j(i+1) \geq B_{min}\}$$

La complessità temporale è $O(k \cdot \text{BatteryLevels})$. Discretizzando la batteria in ~200 livelli con l'ADC, il problema si risolve in frazioni di secondo anche su Arduino, risultando perfetto per una schedulazione giornaliera.

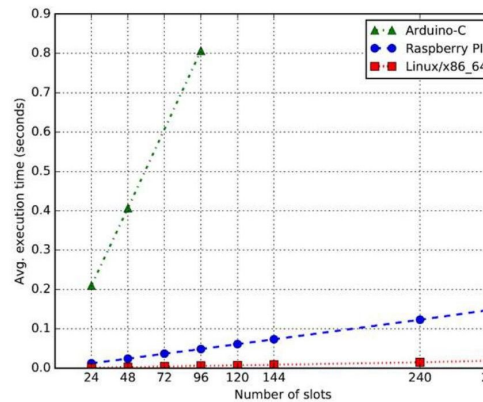


Figura 4.13. Grafico tempo di esecuzione

4.7 Riepilogo

Domanda – Possibili domande d'esame

- Qual è la differenza tra architettura Harvest-Use e Harvest-Store-Use? - Scrivi le equazioni di conservazione dell'energia per un buffer non ideale. - Classifica le sorgenti per controllabilità e prevedibilità.
- Come si stima la carica via ADC? Derivare la formula. - Quali sono le tre condizioni sufficienti del Teorema di Kansal per l'energy neutrality? - Descrivi l'algoritmo di Kansal nei casi di sovrapproduzione e sottoproduzione con un esempio. - Descrivi la funzione utility $u(dc)$ in Kansal e calcola α, β . - Cos'è il modello a task, come migliora Kansal e come risolve la complessità NP-Hard (Dynamic Programming)?

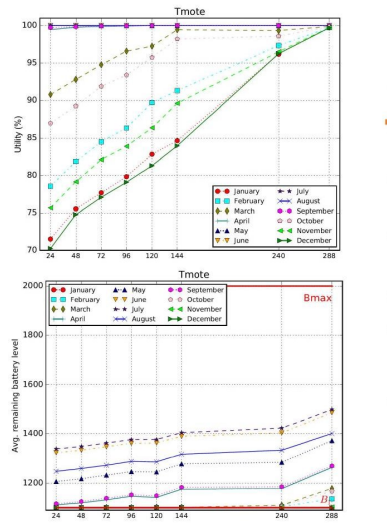


Figura 4.14. Utility e livello medio batteria

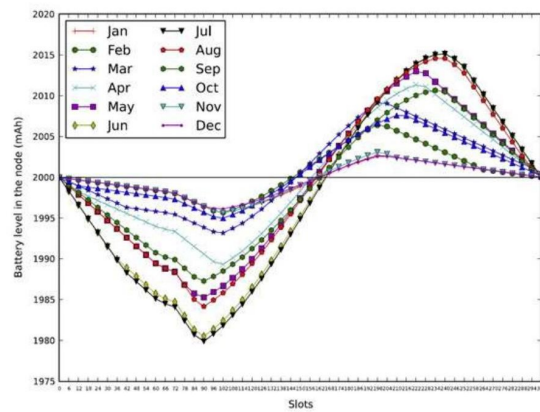


Figura 4.15. Livello batteria durante il giorno

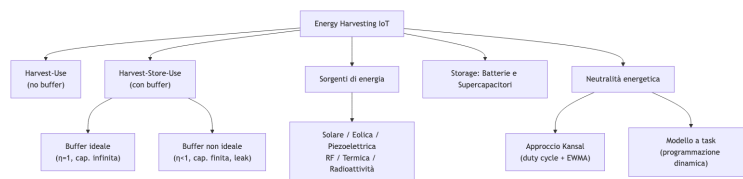


Figura 4.16. Mappa concettuale complessiva dei concetti relativi all'Energy Harvesting e neutralità energetica

Capitolo 5

05 - Preparazione Esame Orale

Documento di preparazione all'esame orale del corso **Mobile and Cyber-Physical Systems**, modulo del Prof. Stefano Chessa. Per ogni schema del documento originale viene fornita una risposta dettagliata, corredata dell'immagine di riferimento e dei link alle lezioni pertinenti.

5.1 Interoperabilità

At the oral exam I will start the exam asking you to comment one of these schemas or to answer to one of these questions:

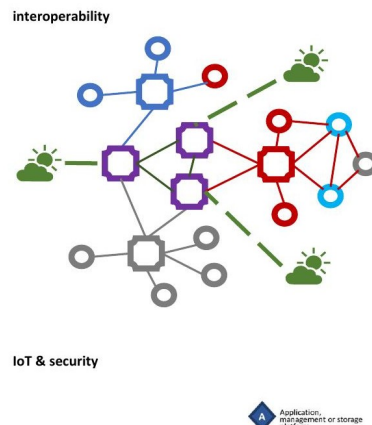


Figura 5.1. Rete IoT eterogenea: dispositivi di produttori diversi (colori diversi) comunicano attraverso gateway di integrazione.

Lo schema mostra una rete in cui dispositivi appartenenti a ecosistemi diversi (rappresentati con colori diversi) devono comunicare tra loro. Il problema fondamentale è l'**interoperabilità**.

Il percorso storico delle soluzioni IoT “dal basso” porta spesso alla creazione di **vertical silos**: sistemi proprietari in cui i dispositivi di un fornitore comunicano solo con l'infrastruttura dello stesso produttore. Questa strategia risponde a un preciso modello di business chiamato **vendor lock-in**, che costringe il cliente a restare nell'ecosistema del fornitore pena una costosa migrazione.

La soluzione naturale è l'adozione di **standard condivisi** (Wi-Fi, ZigBee, Bluetooth, ecc.), ma la proliferazione degli standard sposta il problema al livello middleware e applicativo: oggi coesistono MQTT, CoAP, LWM2M e molti altri protocolli applicativi, incompatibili tra loro.

Per gestire questa eterogeneità si ricorre agli **application-level gateway**, che non si limitano a tradurre protocolli di basso livello ma mappano comportamenti applicativi differenti, operando come interpreti semantici tra ecosistemi.

Le architetture IoT si classificano in quattro tipi:

Tipo	Fornitore	Protocollo	Gateway?
A	Unico	Unico	No
B	Multiplo	Unico condiviso	No / minimale
C	Multiplo	Diversi	Sì — traduzione
D	Multiplo	Eterogenei distribuiti	Sì — gateway multipli

All'aumentare della complessità da A a D, il gateway deve gestire un numero esponenziale di mappature tra protocolli.

Nota – Riferimento

[[Lezione 3 - Interoperabilità e Standard]]

5.2 IoT & Security

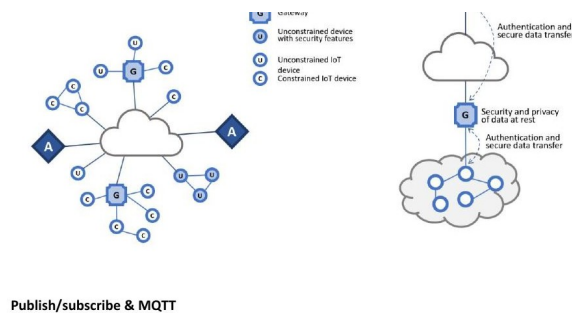


Figura 5.2. Architettura di sicurezza IoT: dispositivi vincolati (C), gateway (G), applicazioni (A) e relative misure di sicurezza per ogni livello.

Lo schema mostra un'architettura IoT a strati in cui la sicurezza deve essere garantita a ogni livello: tra dispositivi periferici e gateway (autenticazione + trasferimento sicuro), e tra gateway e cloud/applicazione (sicurezza dei dati a riposo + autenticazione).

I dispositivi IoT sono spesso sistemi embedded economici, prodotti con forti incentivi a ridurre costi e tempi di immissione sul mercato, a discapito della sicurezza. La conseguenza sono centinaia di milioni di dispositivi vulnerabili, privi di meccanismi di patching efficaci.

La raccomandazione **ITU-T Y.2066** definisce tre aree fondamentali di sicurezza:

1. **Sicurezza della comunicazione** — riservatezza e integrità dei dati in transito tra dispositivi e piattaforme.
2. **Sicurezza della gestione dei dati** — protezione di riservatezza e integrità quando i dati sono archiviati o elaborati (*data at rest*).
3. **Sicurezza della fornitura del servizio** — prevenzione di accessi non autorizzati e protezione della privacy degli utenti.

A questi si aggiungono l'**autenticazione mutua** (entrambe le parti si verificano reciprocamente, più robusta dell'autenticazione a una via) e la capacità di audit di sicurezza.

Il **gateway** gioca un ruolo centrale: - Gestisce identificazione e autenticazione di ogni dispositivo connesso. - Protegge la privacy dei dispositivi periferici. - Supporta manutenzione, aggiornamento firmware e autodiagnosi remota. - Applica policy di configurazione dinamiche.

Attenzione – Dispositivi vincolati e limiti

I *constrained devices* spesso non dispongono di hardware crittografico dedicato, rendendo impraticabile la cifratura dei dati archiviati. Con il *Massive IoT*, la privacy diventa critica per la grande mole di dati sensibili (medici, posizione, abitudini) raccolti.

Nota – Riferimento

[[Lezione 3 - Interoperabilità e Standard]] (sezione sicurezza)

5.3 Publish/Subscribe & MQTT

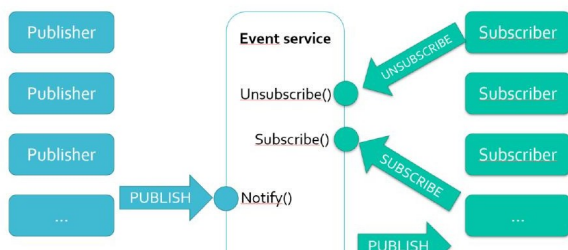


Figura 5.3. Il paradigma publish/subscribe: i publisher inviano messaggi al broker tramite *PUBLISH*; i subscriber si registrano (*SUBSCRIBE*) e ricevono notifiche (*PUBLISH* dal broker); possono anche disdire (*UNSUBSCRIBE*).

Il paradigma **publish/subsribe** è il cuore di MQTT. A differenza del modello client/server, il pub/sub è *loosely coupled* grazie a tre forme di disaccoppiamento:

- **Space decoupling:** publisher e subscriber non si conoscono, non condividono IP né porta.
- **Time decoupling:** non devono essere attivi contemporaneamente.
- **Synchronization decoupling:** le operazioni non sono bloccanti.

Il **broker** è il server dell'infrastruttura: riceve tutti i messaggi, li filtra per topic, li distribuisce ai subscriber interessati. Le operazioni fondamentali sono quattro: *Publish*, *Subscribe*, *Notify*, *Unsubscribe*.

MQTT (*Message Queuing Telemetry Transport*) implementa pub/sub con filtraggio basato su **topic** — stringhe gerarchiche separate da / (es. `home/firstfloor/bedroom/temperature`). I subscriber possono usare wildcard: - `+` sostituisce un singolo livello: `home/+/temperature` - `#` sostituisce tutti i livelli successivi: `home/#`

MQTT opera su TCP (porta 1883, o 8883 con TLS) e concentra la complessità sul broker, mantenendo i client semplici. È *data agnostic*: il payload può essere binario, testo, JSON o XML.

Nota – Riferimento

[[Lezione 4 - MQTT Part 1]]

5.4 MQTT – II (QoS 2)

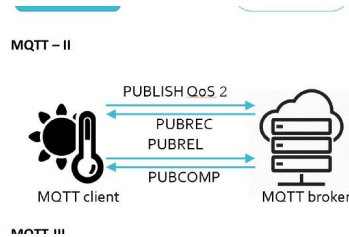


Figura 5.4. Handshake a quattro fasi del QoS 2 tra MQTT client e broker: *PUBLISH* → *PUBREC* → *PUBREL* → *PUBCOMP*.

MQTT definisce tre livelli di **Quality of Service**:

Livello	Nome	Garanzia	Meccanismo
QoS 0	At most once	Nessuna	Nessun ACK
QoS 1	At least once	Almeno una consegna	PUBACK (possibili duplicati)
QoS 2	Exactly once	Esattamente una consegna	4-way handshake

QoS 2 è il livello più affidabile. Il handshake a quattro fasi serve a evitare sia la perdita che la duplicazione:

1. **PUBLISH**: il client invia il messaggio al broker (con packetId).
2. **PUBREC** (*Publish Received*): il broker conferma la ricezione e memorizza il messaggio.
3. **PUBREL** (*Publish Release*): il client autorizza il broker a procedere con la consegna effettiva.
4. **PUBCOMP** (*Publish Complete*): il broker conferma che la consegna ai subscriber è avvenuta e il messaggio può essere eliminato.

Il PUBREC garantisce che il messaggio non vada perso; PUBREL + PUBCOMP garantiscono che non venga consegnato due volte. Due fasi non sarebbero sufficienti perché, se il broker consegnasse immediatamente al PUBLISH senza aspettare PUBREL, e il client ritrasmettesse credendo di non aver ricevuto PUBREC, si avrebbero duplicati.

Nota – Riferimento

[[Lezione 4 - MQTT Part 1]]

5.5 MQTT-III — Topic, Sessioni, Retained Messages, Last Will & Keep Alive

5.5.1 Come il broker filtra i messaggi e cosa sono i topic

Il broker usa il **filtraggio basato su topic** (topic-based filtering): ogni messaggio PUBLISH porta un topic; il broker lo confronta con le sottoscrizioni attive e lo consegna ai subscriber che corrispondono. I topic sono stringhe gerarchiche; le wildcard + e # permettono iscrizioni a insiemi di topic. Il broker non ispeziona il payload (a differenza del content-based filtering), il che consente la cifratura end-to-end.

5.5.2 Sessioni Persistenti

Quando un dispositivo IoT si disconnette (per sleep, copertura persa, reset), rischia di perdere le sottoscrizioni attive e i messaggi pubblicati durante l'assenza. Le **persistent session** risolvono questo: impostando `cleanSession = false` nel CONNECT, il broker conserva per quel `clientId`: - Tutte le sottoscrizioni attive - I messaggi non consegnati con QoS 1 o 2 - I messaggi in attesa di completamento del flusso QoS 2

Alla riconnessione con lo stesso `clientId`, la sessione viene ripristinata automaticamente. Il broker segnala la presenza di una sessione precedente tramite il flag *Session Present* nel CONNACK.

5.5.3 Retained Messages

Nel pub/sub classico, un nuovo subscriber non sa quando arriverà il primo messaggio. I **retained message** risolvono questo: un messaggio pubblicato con `retainFlag = true` viene conservato dal broker (uno per topic). Ogni nuovo subscriber riceve immediatamente l'ultimo messaggio trattenuto al momento dell'iscrizione, senza aspettare la prossima pubblicazione.

Tip – Pattern potente

Un dispositivo pubblica il proprio stato ("ON") come retained message. Configura anche un testamento con payload "OFF" e retain flag attivo sullo stesso topic. Se va in crash, il broker pubblica automaticamente "OFF" come retained message — tutti i client (connessi e futuri) vedono lo stato corretto.

5.5.4 Last Will & Testament

Quando un dispositivo si disconnette *normalmente* (DISCONNECT), può notificare gli altri esplicitamente. Per le **disconnessioni anomale** (crash, timeout, interruzione di rete) non esiste tale possibilità. Il **Last Will & Testament** consiste in un messaggio pre-configurato consegnato al broker al momento del CONNECT: se il broker rileva una disconnessione anomala, pubblica quel messaggio automaticamente. Il testamento ha topic, payload, QoS e retained flag propri.

Il broker invia il testamento quando: errore di I/O sulla connessione, mancato invio di PINGREQ entro il keep alive, chiusura brusca TCP senza DISCONNECT, o chiusura forzata per errore di protocollo. Se il client si disconnette con DISCONNECT regolare, il testamento viene scartato.

5.5.5 Keep Alive

TCP può mantenere una connessione apparentemente attiva anche quando il peer è irraggiungibile. Il **Keep Alive** risolve: il client dichiara un intervallo (campo a 16 bit nel CONNECT) entro cui si impegna a inviare almeno un pacchetto. Se non lo fa, il broker chiude la connessione e invia il testamento. Il client invia PINGREQ se non ha altro traffico; il broker risponde con PINGRESP. Il valore 0 disabilita il meccanismo.

Nota – Riferimento

[[Lezione 4 - MQTT Part 1]], [[Lezione 8 - MQTT Part 2]]

5.6 ZigBee - I (Join through Association)

Lo schema mostra il processo con cui un dispositivo si unisce a una rete ZigBee esistente attraverso il meccanismo di **associazione**. Si tratta di un'interazione a tre livelli (Application, Network, MAC) tramite service primitives.

La sequenza è:

1. **Application layer** emette `NETWORK-DISCOVERY.request` al Network Layer.
2. **Network Layer** emette `SCAN.request` al MAC: viene eseguito un active scan per trovare le PAN disponibili.
3. **MAC** esegue lo scan e risponde con `SCAN.confirm`.
4. **Network Layer** risponde all'applicazione con `NETWORK-DISCOVERY.confirm`, che elenca le PAN trovate.
5. **Application layer** sceglie una PAN e invia `JOIN.request` al Network Layer.

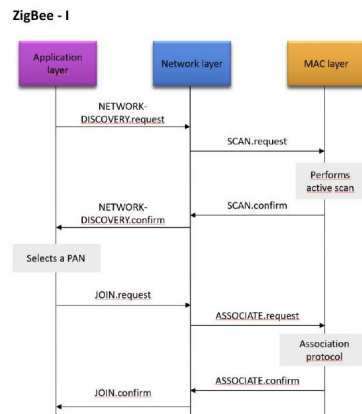


Figura 5.5. Sequenza di primitives tra Application Layer, Network Layer e MAC Layer durante il processo di join di un dispositivo ZigBee.

6. **Network Layer** seleziona un nodo genitore P e avvia il protocollo di associazione MAC tramite **ASSOCIATE.request**.
7. **MAC** esegue il protocollo di associazione: il coordinatore/router assegna un **indirizzo breve a 16 bit** al nuovo dispositivo.
8. **MAC** risponde con **ASSOCIATE.confirm** → **Network Layer** risponde con **JOIN.confirm** all'applicazione.

Nelle reti a stella, P è necessariamente il coordinatore. Nelle reti ad albero o mesh, P può essere qualsiasi router raggiungibile.

Nota – Riferimento

[[Lezione 9 - The ZigBee Standard Part 1]]

5.7 ZigBee – II (Application Layer Stack)

ZigBee aggiunge sopra IEEE 802.15.4 (PHY + MAC) due livelli: **Network Layer** e **Application Layer**. Il livello applicativo si compone di tre elementi:

Application Framework: ospita fino a 240 **Application Objects (APO)**, ciascuno associato a un endpoint (numerato da 1 a 240). Ogni APO è identificato univocamente dalla coppia **<indirizzo di rete, endpoint>**. Più APO sullo stesso dispositivo possono corrispondere ad applicazioni diverse che operano simultaneamente.

ZigBee Device Object (ZDO): occupa l'endpoint 0 ed è la componente gestionale del nodo. Fornisce: - Device & Service Discovery (trova altri nodi e i loro servizi) - Binding Management (crea/modifica le binding table dell'APS) - Network Management (gestisce join/leave e routing table)

Application Support Sublayer (APS): fa da intermediario tra APO/ZDO e il Network Layer. Fornisce: - *Data Service*: trasmissione messaggi con acknowledgment end-to-end - *Binding Service*: creazione di connessioni logiche tra endpoint - *Group Management*: indirizzamento multicast tramite Group Table

L'APS filtra i pacchetti per endpoint e profile ID, scartando quelli non destinati ad applicazioni registrate.

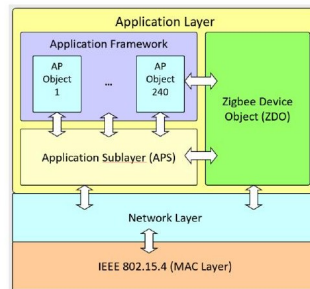


Figura 5.6. Stack protocol ZigBee: dall'IEEE 802.15.4 (MAC) fino all'Application Layer, con Application Framework, ZDO e APS.

Nota – Riferimento

[[Lezione 9 - The ZigBee Standard Part 1]], [[Lezione 12 - Application Layer of ZigBee]]

5.8 ZigBee - III (Topologie di Rete)

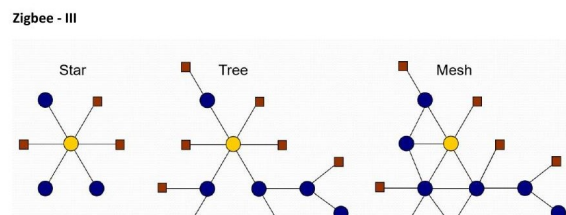


Figura 5.7. Le tre topologie ZigBee: Star (sinistra), Tree (centro), Mesh (destra). Nodo giallo = coordinatore, nodi blu = router, nodi rossi = end-device.

ZigBee supporta tre topologie di rete gestite dal Network Layer:

Star (Stella): tutti i dispositivi comunicano direttamente con il coordinatore. Si mappa direttamente sulla struttura IEEE 802.15.4 e supporta il meccanismo del superframe/beacon. Semplice ma non scalabile: ogni dispositivo deve essere nel raggio radio del coordinatore.

Tree (Albero): il coordinatore è la radice, i router sono nodi interni, gli end-device sono foglie. Può usare il superframe. Il routing segue la struttura gerarchica (tree routing): i pacchetti salgono verso la radice e poi scendono verso la destinazione. Il vantaggio è la semplicità; lo svantaggio è la rigidità e i percorsi spesso subottimali.

Mesh: topologia più flessibile in cui i router possono comunicare direttamente tra loro indipendentemente dalla struttura ad albero. Usa mesh routing (ispirato ad AODV): se non esiste un'entry nella routing table,

viene avviato il Route Discovery Protocol (broadcast RREQ, unicast RREP). Non supporta il beaconing. È più robusto: percorsi alternativi sono disponibili se un link cade.

Tip – Coesistenza

Tree routing e Mesh routing possono coesistere nella stessa rete: ogni router mantiene sia la logica ad albero che la routing table mesh, passando dinamicamente dall'uno all'altro.

Nota – Riferimento

[[Lezione 9 - The ZigBee Standard Part 1]]

5.9 ZigBee - IV (Indirizzamento ad Albero)

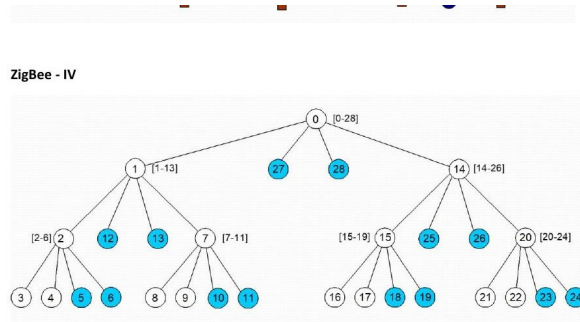


Figura 5.8. Albero di indirizzi ZigBee con $R_m = 2$, $D_m = 2$, $L_m = 3$: ogni nodo gestisce un intervallo di indirizzi calcolato con la formula C_{skip} .

ZigBee assegna gli indirizzi di rete in modo completamente decentralizzato usando la struttura ad albero. Il coordinatore viene configurato con tre parametri: - R_m : massimo numero di router figli per nodo - D_m : massimo numero di end-device figli per nodo - L_m : profondità massima dell'albero

La dimensione del blocco di indirizzi assegnato a un router a profondità d è:

$$C_{skip}(d) = \begin{cases} 1 & \text{se } d = L_m \\ 1 + R_m \cdot C_{skip}(d+1) + D_m & \text{altrimenti} \end{cases}$$

Se un router ha indirizzo A e si trova a profondità d , i suoi figli router ricevono: $A+1$, $A+1+C_{skip}(d+1)$, $A+1+2 \cdot C_{skip}(d+1)$, ... Gli end-device ricevono gli indirizzi successivi a tutti i blocchi router.

Tip – Nell'immagine: $R_m=2$, $D_m=2$, $L_m=3$

- $C_{skip}(3) = 1$, $C_{skip}(2) = 5$, $C_{skip}(1) = 13$, $C_{skip}(0) = 29$
- Coordinatore (addr=0) gestisce [0-28]; primo figlio router addr=1 gestisce [1-13]; secondo figlio router addr=14 gestisce [14-26]; end-device 27 e 28.

Vantaggio: completamente decentralizzato, nessun rischio di collisione, nessun coordinamento distribuito necessario.

Svantaggio: rigido — se un sottoalbero è saturo, nuovi dispositivi non possono aggiungersi tramite quel nodo anche se ce ne fosse necessità.

Nota – Riferimento

[[Lezione 9 - The ZigBee Standard Part 1]]

5.10 ZigBee - V (Tabella APS / Binding Table)

ZigBee - V

Src Addr (64 bits)	Src EP	Cluster ID	Dest Addr (16/64 bits)	Addr/Grp	Dest EP
0x3232...	5	0x0006	0x1234...	A	12
0x3232...	6	0x0006	0x796F...	A	240
0x3232...	5	0x0006	0x9999	G	-
0x3232...	5	0x0006	0x5678...	A	44

Figura 5.9. Esempio di APS Binding Table: ogni entry mappa un endpoint sorgente con cluster e destination address/endpoint.

La tabella mostra la **APS Binding Table**, il meccanismo con cui ZigBee implementa l'**indirizzamento indiretto**. Un dispositivo sorgente (che conosce solo il proprio endpoint e il cluster di interesse) non ha bisogno di conoscere l'indirizzo di rete del destinatario: consulta la binding table.

Le colonne della tabella sono: - **Src Addr (64 bit)**: indirizzo MAC a 64 bit del dispositivo sorgente. - **Src EP**: endpoint sorgente (1-240). - **Cluster ID**: il cluster di riferimento (es. 0x0006 = OnOff Cluster). - **Dest Addr (16/64 bit)**: indirizzo di destinazione (short 16 bit o MAC 64 bit). - **Addr/Grp**: **A** = indirizzo unicast, **G** = indirizzo di gruppo (multicast). - **Dest EP**: endpoint destinazione (vuoto per i gruppi).

Nell'esempio, il dispositivo 0x3232... endpoint 5 può inviare messaggi: - All'indirizzo 0x1234... endpoint 12 (unicast) - All'indirizzo 0x796F... endpoint 240 (unicast) - Al gruppo 0x9999 (multicast, nessun endpoint specifico) - All'indirizzo 0x5678... endpoint 44 (unicast)

Questo meccanismo è fondamentale: se un dispositivo cambia indirizzo di rete (dopo un reset), l'APS usa la **Address Map Table** (che associa indirizzi 16-bit agli immutabili MAC 64-bit) per ripristinare i binding automaticamente.

Nota – Riferimento

[[Lezione 12 - Application Layer of ZigBee]]

5.11 ZigBee - VI (Cluster e Binding tra Dispositivi)

Lo schema illustra come il **binding** ZigBee colleghi dispositivi tramite cluster. Ogni rettangolo contiene le lettere C (Client) o S (Server) per ciascun cluster supportato.

Un **Cluster** è una collezione di comandi e attributi che definisce l'interfaccia per una funzionalità specifica. Ogni cluster ha un ID a 16 bit (es. 0x0006 = OnOff, 0x0008 = Level Control). Il cluster lato **server** ospita lo stato (attributi); il lato **client** invia comandi.

Nell'esempio: - La **Configuration tool** (solo C) si lega all'**On/off switch** (S+C) per configurarlo. - L'**On/off switch** (come client) controlla la **Simple lamp** (S = solo server OnOff). - Il **Dimmer switch** (S+C) riceve configurazioni (S) e controlla la **Dimmable lamp** che implementa sia OnOff (S) che Level Control (S).

Il binding è unidirezionale: si va da client a server. Viene creato tramite **BIND.request** allo ZDO e memorizzato nella binding table dell'APS. Un singolo client può essere legato a più server (fan-out).

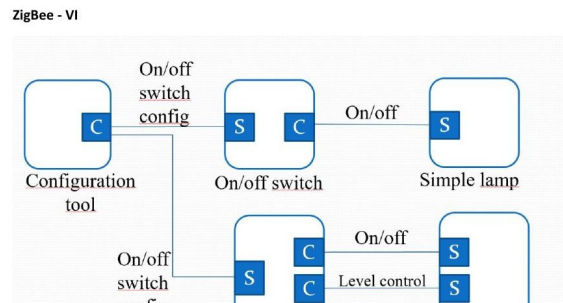


Figura 5.10. Esempio di binding ZigBee: Configuration tool configura un on/off switch, che controlla una Simple lamp e un Dimmer switch, il quale controlla una Dimmable lamp.

Nota – Riferimento

[[Lezione 12 - Application Layer of ZigBee]]

5.12 Duty Cycle - I (Calcolo del Duty Cycle)

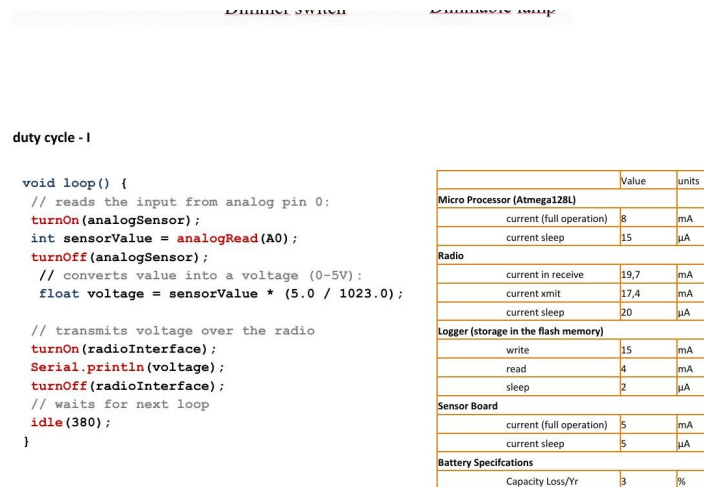


Figura 5.11. Codice Arduino che implementa il duty cycling selettivo dei componenti (sensore, radio) e tabella dei consumi in mA per componente e stato.

Il **duty cycle** di un componente è la frazione di tempo in cui è attivo rispetto al periodo totale. La strategia fondamentale di risparmio energetico nei dispositivi IoT è attivare ogni componente **solo quando strettamente necessario**.

Il codice mostra il pattern classico:

turnOn(analogSensor) → sensore acceso solo durante lettura

```

analogRead(A0)          → lettura
turnOff(analogSensor)   → sensore spento

turnOn(radioInterface) → radio accesa solo durante trasmissione
Serial.println(voltage)
turnOff(radioInterface)

idle(380)                → MCU in sleep 380ms

```

Usando i valori della tabella (es. Tmote Sky): - Processore attivo: 8 mA, sleep: 15 A - Radio TX: 17.4 mA, RX: 19.7 mA, sleep: 20 A - Sensore: 5 mA, sleep: 5 A

Se le operazioni di sensing + TX durano ~20 ms e il ciclo totale è 400 ms, il duty cycle della radio è $20/400 = 5\%$. La corrente media risultante è molto inferiore alla corrente di picco, estendendo la vita della batteria di ordini di grandezza.

La perdita di capacità della batteria del 3%/anno indica che anche con battery standby c'è un degrado nel tempo.

Nota – Riferimento

[[Lezione 23 - Embedded Programming e Arduino]]

5.13 Duty Cycle - II (Impatto sulla Vita della Batteria)

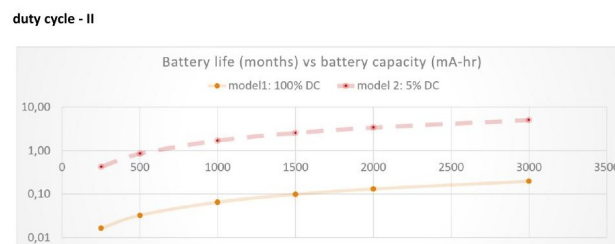


Figura 5.12. Grafico log-log: vita della batteria (mesi) vs capacità della batteria (mAh) per duty cycle 100% (model1) e 5% (model2).

Il grafico mostra l'impatto del duty cycle sulla vita della batteria in scala logaritmica. Due modelli a confronto:

- **Model 1 (100% DC):** il dispositivo è sempre attivo. La vita della batteria scala linearmente con la capacità ma rimane nell'ordine dei mesi (0.01–0.3 mesi per capacità 500–3000 mAh).
- **Model 2 (5% DC):** il dispositivo è attivo solo il 5% del tempo. La vita si estende di un fattore ~20: con la stessa batteria, si passa da 0.1 a circa 2–8 mesi.

La scala logaritmica rivela che la relazione vita-capacità non è lineare: il duty cycle agisce moltiplicativamente. Ridurre il duty cycle da 100% a 5% equivale a moltiplicare la capacità effettiva della batteria per 20.

Conclusione progettuale: per applicazioni IoT con autonomia di anni, ridurre il duty cycle è molto più efficace che aumentare la capacità della batteria. Una batteria da 3000 mAh a 5% DC dura circa 8 mesi; per durare anni si deve abbassare ulteriormente il duty cycle o usare energy harvesting.

Nota – Riferimento

[[Lezione 23 - Embedded Programming e Arduino]]

5.14 Embedded Programming - I (Modello Arduino)

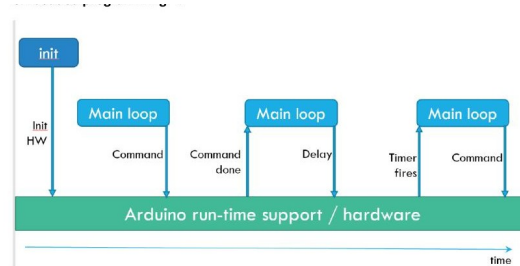


Figura 5.13. Modello di esecuzione Arduino: `init` → `loop()` ripetuto. I comandi attivano l'hardware; `delay()` aspetta; il timer fires avvia la ripetizione.

Lo schema mostra il **modello di programmazione Arduino**, basato su un singolo thread di esecuzione:

1. **init:** la funzione `setup()` viene eseguita una sola volta all'avvio. Inizializza l'hardware, configura i pin, prepara le comunicazioni seriali.
2. **Main loop:** la funzione `loop()` viene invocata ripetutamente all'infinito dal runtime Arduino.
3. **Command:** dentro `loop()`, il codice interagisce con l'hardware tramite comandi sincroni (es. `analogRead()`, `Serial.println()`).
4. **Delay:** `delay(ms)` blocca l'esecuzione per il tempo specificato (busy waiting o sleep), poi il timer fires riavvia il loop.

Il modello è semplice ma **bloccante**: durante `delay()` il processore non può fare altro. Gli interrupt possono intervenire anche durante il delay (come mostrato nel codice Arduino interrupt), ma il thread principale rimane sospeso.

Differenza con TinyOS: nel modello Arduino tutto avviene sequenzialmente in un unico thread. In TinyOS (modello event-driven) il sistema reagisce ad eventi: non c'è un loop esplicito, ma handler che si concatenano tramite eventi hardware.

Nota – Riferimento

[[Lezione 23 - Embedded Programming e Arduino]]

5.15 Embedded Programming - II (Modello TinyOS/Event-Driven)

TinyOS usa un modello **event-driven** con componenti e interfacce. Non esiste un loop bloccante: il sistema è guidato dagli eventi hardware. Lo schema mostra una catena tipica:

1. **init:** configura timer e hardware.
2. **Timer handler** (evento): scatta quando il timer fires → avvia una lettura dal sensore (**Start read**).
3. **Read handler** (evento): scatta quando la lettura è completata (**Read done**) → posta un task per il processing.
4. **task:** elabora i dati (*Data processing happens here*) → avvia la trasmissione radio.

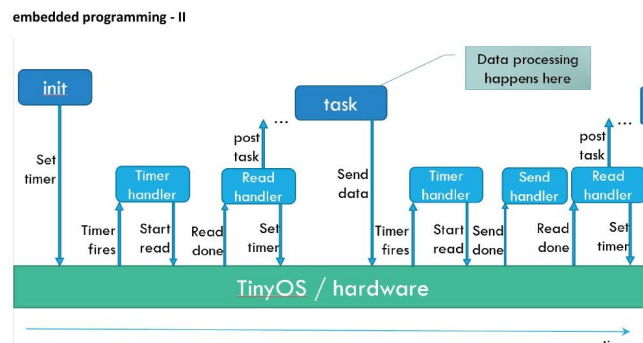


Figura 5.14. Modello di esecuzione TinyOS: catena di eventi (Timer → Read → task → Send) senza loop bloccante. Il data processing avviene nel task asincrono.

5. **Send handler** (evento): scatta quando la trasmissione è completa → riconfigura il timer per il prossimo ciclo.

Vantaggi del modello event-driven: - Nessun busy waiting: il processore dorme tra un evento e l'altro. - Nessuno stack multiplo: un unico stack (run-to-completion semantics). - Efficienza energetica: il MCU è attivo solo durante gli handler.

Confronto con Arduino: Arduino è più semplice da programmare (loop sequenziale), ma meno efficiente in termini energetici. TinyOS è ottimale per nodi a basso consumo (mica motes, TMote Sky) dove ogni microsecondo di sleep conta.

Nota – Riferimento

[[Lezione 23 - Embedded Programming e Arduino]]

5.16 Arduino - Interrupt

Il codice mostra il meccanismo degli **interrupt hardware** su Arduino:

```
volatile int count = 0;
attachInterrupt(0, interruptSwitchGreen, RISING);
```

`attachInterrupt(0, handler, RISING)` collega il pin 2 (interrupt 0 su Arduino Uno) alla funzione `interruptSwitchGreen`, eseguita automaticamente sul fronte di salita del segnale.

Punti chiave: - La variabile `count` è dichiarata `volatile`: indica al compilatore di non ottimizzarla in registro, perché può essere modificata da handler fuori dal flusso normale. - Gli interrupt sono ricevuti **anche durante `delay()`** (come nota il commento): il `delay` non disabilita gli interrupt, quindi l'handler può intervenire in qualsiasi momento. - L'interrupt resetta `count=0` e accende il LED verde; il loop principale incrementa `count`, aspetta 1 secondo, e gestisce il timeout a 10.

Utilizzo tipico negli embedded IoT: gli interrupt sono usati per ricevere dati dal sensore (read done), notificare la fine di una trasmissione radio, o rispondere a eventi fisici (pressione di un pulsante) senza polling continuo, risparmiando energia.

Nota – Riferimento

[[Lezione 23 - Embedded Programming e Arduino]]

```

Arduino - interrupt
/**
 * test interrupt
 */

volatile int greenLed=7;
volatile int count=0;

void setup()
{
  Serial.begin(9600);
  pinMode(greenLed, OUTPUT);

  digitalWrite(greenLed, LOW);

  attachInterrupt(0,
    interruptSwitchGreen,RISING);
  // 0 because it is pin 2
}

void loop() {
  count++;
  delay(1000);
  // interrupts are received
  // also within delay!

  Serial.print("waiting:");
  Serial.println(count);

  if ( count == 10 )
  {
    count = 0;
    digitalWrite(greenLed, LOW);
    Serial.println("now off");
  }
}

void interruptSwitchGreen()
{
  digitalWrite(greenLed, HIGH);
  count=0;
  Serial.print("now on");
}

```

Figura 5.15. Esempio di programmazione con interrupt su Arduino: `attachInterrupt()` collega il pin 2 a `interruptSwitchGreen()`; `count` viene resettato e il LED acceso all'interrupt.

5.17 SMAC (Sensor-MAC)

S-MAC (*Sensor MAC*) è un protocollo MAC per WSN che riduce il consumo energetico tramite **sincronizzazione locale**: nodi vicini si accordano su un periodo di ascolto comune (listen period) e dormono nel resto del tempo.

Meccanismo: 1. Ogni nodo trasmette periodicamente un pacchetto **SYNC** che annuncia il proprio schedule (quando si sveglierà). 2. I vicini che ricevono il SYNC adottano lo stesso schedule o mantengono il proprio e memorizzano quello del vicino. 3. Nel **listen period**: esecuzione di CSMA/CA con RTS/CTS prima di trasmettere. 4. Nel **sleep period**: la radio è spenta.

Duty cycle: definito come $DC = t_{listen} / (t_{listen} + t_{sleep})$, tipicamente 10% in S-MAC.

Problema della latenza multi-hop: come visibile nello schema, ogni nodo deve aspettare il periodo di ascolto del nodo successivo. La latenza totale si accumula: se ogni hop aggiunge un'attesa $\approx t_{sleep}$, un percorso di n hop ha latenza $\approx n \cdot (t_{sleep}/2)$ in media.

Adaptive duty cycle: se un nodo riceve un RTS o CTS (anche non destinato a lui), capisce che c'è traffico nelle vicinanze e mantiene la radio accesa — potrebbe essere il prossimo hop e il messaggio arriverà presto.

Nota – Riferimento

[[Lezione 17 - MAC Protocols]]

5.18 BMAC - I (Struttura del Protocollo)

B-MAC (*Berkeley MAC*) usa un approccio completamente diverso da S-MAC: **nessuna sincronizzazione** tra i nodi. Il principio è il **preamble sampling** (o Low Power Listening, LPL):

Lato ricevitore: - Si sveglia periodicamente ogni t_{check} millisecondi. - Campiona il canale per un breve istante. - Se rileva attività (preamble), rimane sveglio e riceve il pacchetto dati. - Altrimenti, si riaddormenta immediatamente.

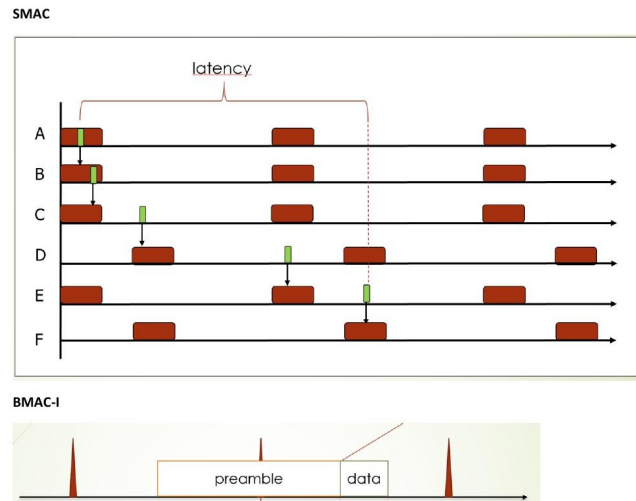


Figura 5.16. S-MAC: i nodi A-F hanno schedule sincronizzati (verde = listen, rosso = active/TX). La latenza multi-hop si accumula: il pacchetto da A deve aspettare i periodi di ascolto di ogni hop successivo.

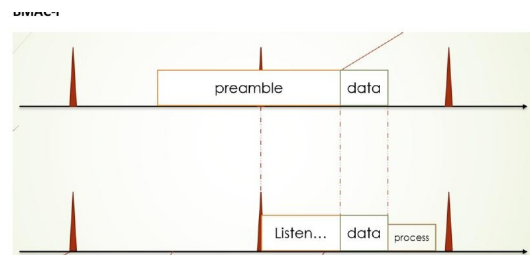


Figura 5.17. B-MAC: il mittente (in alto) invia un lungo preamble seguito dai data; il ricevitore (in basso) si sveglia, ascolta, individua il preamble, rimane sveglio e riceve i data.

Lato mittente: - Quando vuole trasmettere, invia un **preamble lungo** per una durata $> t_{check}$. - Questo garantisce che, qualunque sia il momento in cui il ricevitore si sveglia per campionare, troverà il preamble ancora in corso. - Dopo il preamble, invia i dati effettivi.

Confronto con S-MAC: - B-MAC non richiede sincronizzazione $\rightarrow$ deployment più semplice. - Il preamble lungo consuma energia extra dal mittente. - Il ricevitore non spreca energia in lunghi listen period, ma solo in brevi campionamenti.

Nota – Riferimento

[[Lezione 17 - MAC Protocols]]

5.19 B-MAC - II (Trade-off t_{check} e Vita del Trasmettitore)

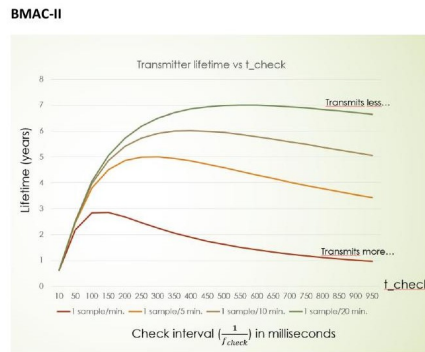


Figura 5.18. Grafico B-MAC: vita del trasmettitore (anni) vs intervallo di check t_{check} (ms), per diverse frequenze di campionamento (1 sample/min, 1/5 min, 1/10 min, 1/20 min).

Il grafico mostra il trade-off fondamentale di B-MAC: come la frequenza di trasmissione e l'intervallo t_{check} influenzano la vita della batteria del **trasmettitore**.

Interpretazione: - Ogni curva rappresenta una diversa frequenza di campionamento dati (1 campione/minuto $\rightarrow$ consumo più alto, 1 campione/20 minuti $\rightarrow$ consumo più basso). - Al crescere di t_{check} (asse x), il preamble deve essere più lungo (per garantire che il ricevitore lo trovi), quindi il trasmettitore consuma di più per ogni trasmissione. - Tuttavia, la vita del trasmettitore ha un **massimo** per un valore ottimale di t_{check} : troppo breve $\rightarrow$ il ricevitore si sveglia spesso ma il preamble è corto (basso overhead); troppo lungo $\rightarrow$ preamble lunghissimo, overhead alto. - Per frequenze di trasmissione più basse (1/20 min), il trasmettitore vive molto più a lungo (fino a 7+ anni).

Conclusione: il parametro t_{check} va ottimizzato in base al profilo di trasmissione dell'applicazione. Non esiste un valore universalmente ottimale.

Nota – Riferimento

[[Lezione 17 - MAC Protocols]]

5.20 X-MAC / B-MAC (Confronto LPL vs X-MAC)

X-MAC migliora B-MAC risolvendo lo spreco energetico del long preamble con i **short preamble strobes** con target address:

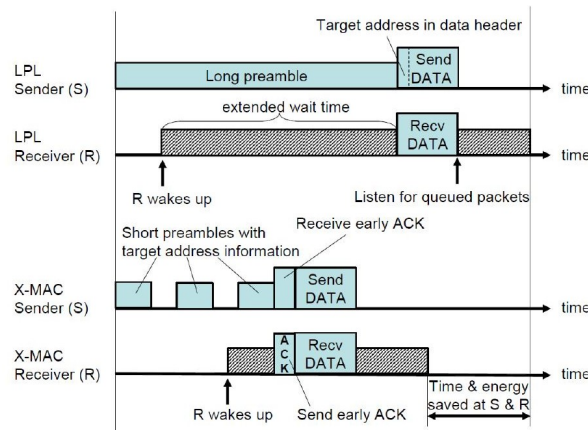


Figura 5.19. Confronto LPL (B-MAC, righe superiori) e X-MAC (righe inferiori): X-MAC usa preamble corti con indirizzo target; il ricevitore invia early ACK; mittente e ricevitore risparmiano tempo ed energia.

LPL (B-MAC): - Mittente: invia un unico preamble lungo $> t_{check}$, poi i dati. - Ricevitore: si sveglia, trova il preamble, rimane sveglio per tutta la durata del preamble + dati. - **Problema:** anche i nodi non destinatari che si svegliano durante il preamble restano svegli inutilmente (overhearing).

X-MAC: - Mittente: invia una sequenza di **short preambles** contenenti ciascuno l'indirizzo del destinatario target. - Ricevitore target: si sveglia, legge il proprio indirizzo nel preamble, invia immediatamente un **early ACK**. - Mittente: riceve l'ACK → interrompe la sequenza di preamble → trasmette i dati direttamente. - **Risparmio:** il mittente non deve completare tutta la sequenza di preamble; il ricevitore non aspetta un preamble lungo. - **Risparmio per i non-destinatari:** vedono il proprio indirizzo assente nel preamble → si riaddormentano subito.

Il risultato è un risparmio sia per il mittente (preamble più corto in media) sia per il ricevitore (less idle listening), indicato nello schema come “Time & energy saved at S & R”.

Nota – Riferimento

[[Lezione 17 - MAC Protocols]]

5.21 IEEE 802.15.4 - I (Struttura del Superframe)

IEEE 802.15.4 definisce una struttura temporale chiamata **superframe** per organizzare l'accesso al canale in reti beacon-enabled:

Beacon frame: il coordinatore trasmette periodicamente un beacon che annuncia l'inizio del superframe, contiene i parametri della rete e la lista dei dispositivi con dati pendenti.

CAP (Contention Access Period): periodo di accesso conteso tramite **CSMA/CA slottato**. Tutti i dispositivi competono per l'accesso al canale. Usato per traffico non deterministico (dati aperiodici).

CFP (Contention Free Period): suddiviso in **GTS (Guaranteed Time Slots)**, assegnati dal coordinatore a specifici dispositivi per comunicazioni deterministiche a bassa latenza. Fino a 7 GTS per superframe. Usato per applicazioni real-time che richiedono garanzie temporali.

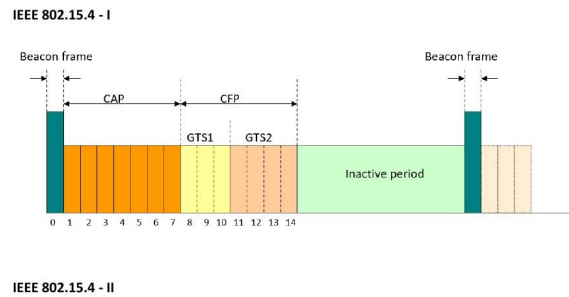


Figura 5.20. Struttura del superframe IEEE 802.15.4: Beacon frame → CAP (Contention Access Period, slot 0–6) → CFP (Contention Free Period, GTS1 e GTS2, slot 7–11) → Inactive period (slot 12–14) → prossimo Beacon.

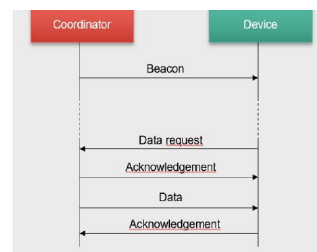
Inactive period: tutti i dispositivi (incluso il coordinatore) possono dormire per risparmiare energia. La durata del periodo inattivo determina il duty cycle della rete.

La durata del superframe è configurabile e determina il duty cycle: un inactive period lungo → basso duty cycle → lunga vita della batteria → latenza più alta.

Nota – Riferimento

[[Lezione 18 - The IEEE 802.15.4 Standard]]

5.22 IEEE 802.15.4 - II (Indirect Data Transfer)



IEEE 802.15.4 - III



Figura 5.21. Trasferimento dati indiretto in IEEE 802.15.4: il dispositivo si sveglia al beacon, invia una Data request al coordinatore, riceve ACK e poi i dati.

Il trasferimento **indiretto** è usato quando il destinatario è un dispositivo che dorme la maggior parte del tempo (RFD, end-device). Il coordinatore non può trasmettere dati in modo asincrono perché il dispositivo potrebbe essere in sleep.

Sequenza: 1. Il **coordinatore** trasmette un **Beacon** che include (nella lista *pending addresses*) l'indirizzo dei dispositivi per cui ha dati in coda. 2. Il **dispositivo** si sveglia al beacon, legge la lista, se vede il proprio indirizzo invia una **Data request** al coordinatore. 3. Il **coordinatore** risponde con un **Acknowledgement** immediato. 4. Il **coordinatore** trasmette i **dati** accodati. 5. Il **dispositivo** risponde con un **Acknowledgement**.

Il device può poi tornare in sleep. Il vantaggio è che il dispositivo non deve rimanere sveglio ad aspettare dati: si sveglia solo al beacon (duty cycle controllato), controlla se ci sono dati per lui, li recupera e dorme di nuovo.

Nota – Riferimento

[[Lezione 18 - The IEEE 802.15.4 Standard]]

5.23 IEEE 802.15.4 - III (Protocollo di Associazione)

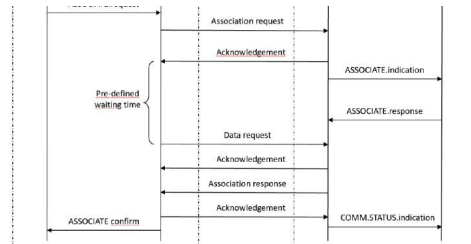


Figura 5.22. Protocollo di associazione IEEE 802.15.4: sequenza di messaggi tra NWK e MAC layer del dispositivo (sinistra) e del coordinatore/router (destra).

Lo schema mostra il protocollo di **associazione** visto a livello di primitives NWK-MAC. È il meccanismo con cui un dispositivo entra nella rete:

1. Il NWK del dispositivo emette **ASSOCIATE.request** al proprio MAC layer.
2. Il MAC invia un **Association request** frame al coordinatore.
3. Il coordinatore MAC risponde con **Acknowledgement** immediato.
4. Il coordinatore NWK riceve **ASSOCIATE.indication** e prepara la risposta con **ASSOCIATE.response**.
5. Il dispositivo aspetta un **pre-defined waiting time** (il coordinatore deve elaborare la richiesta e decidere se accettarla).
6. Il dispositivo MAC invia una **Data request** per recuperare la risposta del coordinatore.
7. Il coordinatore MAC risponde con **Acknowledgement** + **Association response** (contenente l'indirizzo a 16 bit assegnato).
8. Il dispositivo MAC invia **Acknowledgement** finale.
9. Il dispositivo NWK riceve **ASSOCIATE confirm** con l'indirizzo assegnato.
10. Il coordinatore NWK riceve **COMM.STATUS.indication**.

Questo è il meccanismo di basso livello che corrisponde a **ASSOCIATE.request/ASSOCIATE.confirm** nelle primitive ZigBee viste nello schema ZigBee-I.

Nota – Riferimento

[[Lezione 18 - The IEEE 802.15.4 Standard]]

5.24 Energy Harvesting — Domande Generali

Domanda – Domande del PDF

- Explain the difference between an harvest-use and an harvest-store-use architecture
- Explain the classification of sources in terms of controllability and predictability
- Explain the concept of energy neutrality

5.24.1 Harvest-Use vs Harvest-Store-Use

Harvest-Use: l'energia è raccolta e consumata istantaneamente. Non esiste un buffer. Il dispositivo funziona solo se $P_s(t) \geq P_c(t)$ (potenza sorgente > potenza consumata). Se la sorgente produce meno del necessario il dispositivo si spegne; se produce di più l'eccesso va perduto. Esempi: mulini ad acqua, tag RFID passivi.

Harvest-Store-Use: un buffer energetico (batteria ricaricabile o supercapacitore) disaccoppia temporalmente produzione e consumo. L'energia in eccesso ($P_s > P_c$) viene accumulata nel buffer; l'energia in difetto ($P_c > P_s$) viene prelevata dal buffer. Un buffer ideale ha capacità infinita ed efficienza $\eta = 1$. Un buffer reale ha capacità massima B_{max} , efficienza $\eta < 1$, e una potenza di leakage P_{leak} .

5.24.2 Classificazione delle Sorgenti

Le sorgenti si classificano su due assi:

Controllabilità: - *Completamente controllabile:* energia disponibile su richiesta (torcia shake-to-power, sorgenti RF dedicate). - *Parzialmente controllabile:* influenzabile ma non deterministica (RFID in ambiente RF non uniforme). - *Non controllabile:* raccolta solo quando disponibile (sole, vento, calore ambientale).

Prevedibilità (per le sorgenti non controllabili): - *Prevedibile:* esistono modelli affidabili (sole: ciclo giorno/notte + stagioni + meteo). - *Non prevedibile:* nessun modello affidabile (vibrazioni da terremoti).

5.24.3 Concetto di Energy Neutrality

Un dispositivo è **energy neutral** se, in qualsiasi intervallo di tempo, l'energia consumata non supera l'energia raccolta (più la carica iniziale del buffer):

$$\int_0^T P_c(t) dt \leq \int_0^T P_s(t) dt + B_0 \quad \forall T$$

Il raggiungimento dell'energy neutrality richiede di adattare il carico in base alla disponibilità energetica prevista — obiettivo del problema di Kansal.

Nota – Riferimento

[[Lezione 24 - Energy Harvesting IoT]]

5.25 Energy Harvesting - I (Grafico P_s vs P_c)

Il grafico mostra due curve di potenza in funzione del tempo: - $P_s(t)$: potenza prodotta dall'energy harvester (curva decrescente). - $P_c(t)$: potenza consumata dal dispositivo (curva a forma di U).

Le due aree rappresentano:

$$\int_0^T [P_s(t) - P_c(t)]^+ dt$$

Area in blu ($P_s > P_c$, da $t=0$ a $t=4$): energia prodotta in eccesso $\rightarrow$ accumulata nel buffer. Il buffer si carica.

$$\int_0^T [P_c(t) - P_s(t)]^+ dt$$

Area punteggiata ($P_c > P_s$, da $t=4$ a $t=8$): il consumo supera la produzione $\rightarrow$ energia prelevata dal buffer. Il buffer si scarica.

Energy harvesting

- Explain the difference between an harvest-use and an harvest-store-use architecture
- Explain the classification of sources in terms of controllability and predictability
- Explain the concept of energy neutrality

Energy harvesting - I

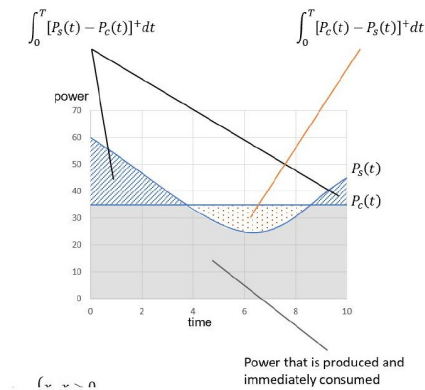


Figura 5.23. Grafico potenza/tempo: l'area tratteggiata in blu ($P_s > P_c$) è l'energia raccolta e accumulata nel buffer; l'area punteggiata ($P_c > P_s$) è l'energia prelevata dal buffer. La retta arancione mostra l'energia immediatamente consumata ($P_c = P_s$).

La retta arancione indica la zona in cui $P_s = P_c$: energia prodotta e immediatamente consumata, senza passare dal buffer.

Il sistema è energy neutral se la prima integrale = seconda integrale su tutto l'orizzonte temporale considerato.

Nota – Riferimento

[[Lezione 24 - Energy Harvesting IoT]]

5.26 Energy Harvesting – II (Stima Stato di Carica)

Energy harvesting – II

platform	v_{min} (volts)	v_{max} (volts)	v_{ref} (volts)	quantization levels (bit)	x_{min}	x_{max}	size of battery (mAh)	x (voltage sampling)	Battery charge (mAh)
RPI	3,5	5	0,004883	10	716	1023	3000	1000	2798
Arduino	7	9	0,008789	10	795	1023	3000	800	359
Tmote	1,8	3	0,000732	12	2457	4095	3000	3277	1652

Figura 5.24. Tabella con parametri di batteria per tre piattaforme (RPI, Arduino, Tmote): tensioni min/max/ref, livelli di quantizzazione, x_{min}/x_{max} e stima Battery charge.

La tabella mostra come misurare lo **stato di carica** (SoC) di una batteria attraverso la sua tensione terminale, per tre piattaforme hardware.

Le colonne chiave: - v_{min} , v_{max} : range operativo della batteria (es. Arduino: 7–9 V). - v_{ref} : risoluzione della quantizzazione ADC (es. Arduino: 0.008789 V per LSB). - **quantization levels (bit)**: risoluzione dell'ADC (10 o 12 bit). - x_{min} , x_{max} : valori ADC corrispondenti a v_{min} e v_{max} . - **Battery charge (mAh)**: capacità totale della batteria stimata.

L'idea è che la tensione ai terminali di una batteria è approssimabile come monotona rispetto alla carica residua. Campionando la tensione tramite ADC e mappando il valore digitale x nell'intervallo $[x_{min}, x_{max}]$, si ottiene una stima lineare della carica rimanente:

$$SoC \approx \frac{x - x_{min}}{x_{max} - x_{min}} \times B_{cap}$$

Questa stima è utile per il task scheduler: sa quanta energia ha disponibile e può pianificare i task di conseguenza (approccio Kansal).

Nota – Riferimento

[[Lezione 24 - Energy Harvesting IoT]]

5.27 Energy Harvesting – III (Approccio Kansal all'Energy Neutrality)

Energy harvesting – III : explain the Kansal approach to energy neutrality

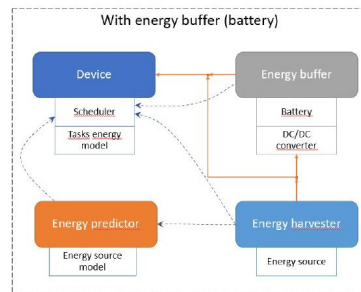


Figura 5.25. Sistema Kansal: il Device (con Scheduler e Tasks energy model) è alimentato dall'Energy buffer (Battery + DC/DC converter), rifornito dall'Energy harvester. L'Energy predictor stima la disponibilità futura e informa lo Scheduler.

L'approccio di **Kansal** all'energy neutrality mira a massimizzare le prestazioni del dispositivo garantendo al contempo che la batteria non si scarichi mai completamente (energy neutral operation).

Il sistema è composto da:

- **Energy harvester**: raccoglie energia dalla sorgente (es. pannello solare).
- **Energy buffer (Battery + DC/DC converter)**: accumula l'energia raccolta e la cede al dispositivo quando necessario.
- **Energy predictor** (con Energy source model): stima la quantità di energia che sarà disponibile nel futuro (ad es. usando previsioni meteo per un pannello solare).
- **Device** con **Scheduler** (e Tasks energy model): pianifica quali task eseguire e a quale frequenza, basandosi sia sullo stato attuale della batteria sia sulla predizione energetica futura.

Idea centrale: adattare il carico (P_c) alla disponibilità energetica prevista. Se le previsioni indicano un giorno soleggiato → il dispositivo può aumentare la frequenza di campionamento. Se le previsioni indicano bassa produzione → il dispositivo riduce l'attività.

Condizione di energy neutrality di Kansal:

$$B_T = B_0 + \eta \int_0^T [P_s - P_c]^+ dt - \int_0^T [P_c - P_s]^+ dt \geq B_{min} \quad \forall T$$

Il Kansal problem è quindi un problema di ottimizzazione: massimizzare le prestazioni (es. frequenza di campionamento) soggetto al vincolo di energy neutrality.

Nota – Riferimento

[[Lezione 28 - Kansal Problem e Energy Neutrality]]

5.28 Energy Harvesting – IV (Task Model per l'Energy Neutrality)

Energy harvesting – IV : explain the task model for energy neutrality

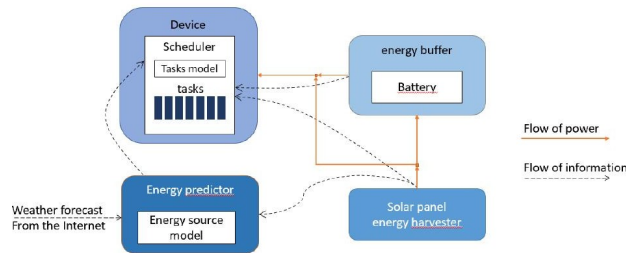


Figura 5.26. Task model: lo Scheduler pianifica i task in base al Tasks model (energia per task) e alle previsioni energetiche dell'Energy predictor (alimentato da weather forecast e da un Solar panel harvester collegato alla Battery).

Lo schema espande il sistema Kansal con il dettaglio del **task model**. Rispetto allo schema III, qui è esplicitato il ruolo delle previsioni meteorologiche esterne.

Componenti:

- **Tasks model:** specifica il costo energetico di ogni task (es. campionamento = X mJ, trasmissione = Y mJ). Il Scheduler usa questo modello per stimare quanta energia consumerà ogni operazione.
- **Scheduler:** dato il livello attuale della batteria e la previsione di energia futura, decide:
 - Quali task eseguire nel prossimo periodo.
 - A quale frequenza/duty cycle eseguirli.
 - Obiettivo: massimizzare la qualità del servizio senza violare l'energy neutrality.
- **Energy predictor + Energy source model:** riceve dati dal weather forecast (Internet) e dalle misurazioni del pannello solare per costruire un modello predittivo della produzione energetica futura.

- **Solar panel energy harvester** → **Battery**: flusso di potenza fisico (linea arancione); informazioni di stato (linea tratteggiata) comunicano il livello di carica allo Scheduler.

Flusso informativo (linee tratteggiate): - Weather forecast → Energy source model → Energy predictor → Scheduler - Battery state → Scheduler (per decisioni in tempo reale) - Tasks model → Scheduler (per la stima del costo)

Nota – Riferimento

[[Lezione 28 - Kansal Problem e Energy Neutrality]]

5.29 Directed Diffusion

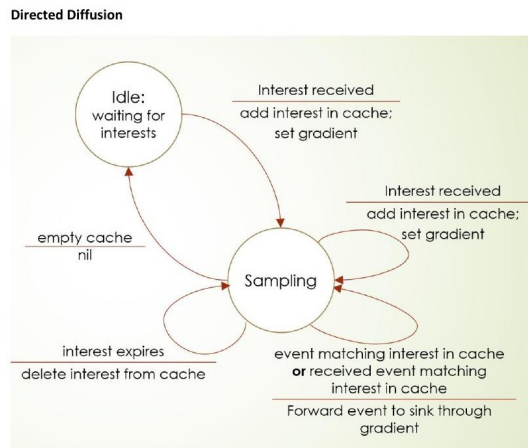


Figura 5.27. Macchina a stati della Directed Diffusion: il nodo è Idle finché non riceve un interest; passa a Sampling e forward gli eventi corrispondenti verso il sink; torna Idle all'expire dell'interest.

Directed Diffusion è un protocollo di routing **data-centric** per WSN, in cui i dati vengono nominati con coppie attributo-valore (non con indirizzi di rete).

Fasi principali:

1. **Interest propagation:** il sink diffonde in broadcast un **interest** (query) che descrive il tipo di dati desiderati (es. **type=temperature, area=sector-A, interval=30s**). Ogni nodo intermedio memorizza l'interest nella propria **interest cache** e imposta un **gradiente** verso il nodo da cui ha ricevuto l'interest (direzione verso il sink).
2. **Sampling** (stato attivo): un nodo che ha interest in cache e rileva dati corrispondenti (o riceve eventi corrispondenti da nodi vicini) li **forwarda verso il sink attraverso il gradiente** impostato.
3. **Expire e ritorno a Idle:** quando l'interest scade (**interest expires**), il nodo lo elimina dalla cache e torna allo stato Idle in attesa di nuovi interest.

La macchina a stati del nodo: - **Idle** (waiting for interests): radio in ascolto, nessun sampling attivo. - **Sampling:** il nodo è in fase di rilevamento e forwarding. - Transizione Idle → Sampling: all'arrivo di un **interest received** → cache interest, set gradient. - Transizione Sampling → Idle: **interest expires** → delete from cache. - Permanenza in Sampling: se arriva un nuovo interest → aggiorna cache e gradiente. - Evento in Sampling: se **event matching interest in cache** → forward al sink tramite gradiente.

Reinforcement: il sink può rinforzare il percorso migliore inviando un interest più frequente lungo il path ottimale, aumentando la frequenza di ricezione dati da quel percorso.

5.30 GPSR - I (Greedy Forwarding e Void)

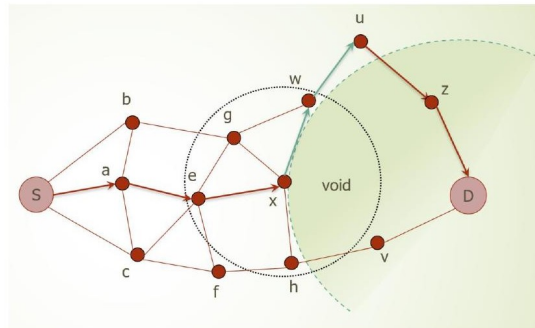


Figura 5.28. GPSR greedy: S forwarda verso x (più vicino a D); x incontra un void (nessun vicino è più vicino a D di x stesso) → attiva il perimeter routing attorno al void per raggiungere D.

GPSR (*Greedy Perimeter Stateless Routing*) è un protocollo di routing geografico per WSN che usa le posizioni fisiche dei nodi. Ogni nodo conosce la propria posizione GPS e quella dei propri vicini immediati.

Greedy Forwarding: - Ogni nodo forward il pacchetto al vicino **più vicino geograficamente alla destinazione D**. - Esempio: $S \rightarrow a \rightarrow e \rightarrow x$, perché in ogni hop si seleziona il vicino con distanza minima da D. - Efficiente quando esiste sempre un vicino più vicino alla destinazione.

Il problema del Void: - Un nodo x si trova in un **void** quando nessuno dei suoi vicini è geograficamente più vicino a D di x stesso. - Il greedy routing si blocca in un void. - Nell'immagine: x è circondato da un void (area verde chiaro) → nessun vicino è più vicino a D.

Soluzione — Perimeter Routing: - Quando si incontra un void, GPSR passa al **perimeter routing** (face routing). - Il nodo x inizia a navigare il confine del void usando la **right-hand rule** sul grafo planare. - Il perimeter routing termina quando si raggiunge un nodo più vicino a D rispetto al punto in cui si è entrati nel void. - Poi si torna al greedy forwarding.

5.31 GPSR - II (Perimeter Routing / Face Traversal)

Il **perimeter routing** di GPSR è basato sulla **right-hand rule** applicata a un grafo planare:

Right-hand rule: muovendo verso il prossimo hop, gira sempre verso l'arco più a destra disponibile rispetto alla direzione di provenienza. Questo garantisce di attraversare ogni faccia del grafo esattamente una volta.

Nell'esempio: - Il pacchetto entra in modalità perimeter a x (che è nel void). - Naviga seguendo la right-hand rule: $x \rightarrow y \rightarrow z \rightarrow \text{destination}$, con possibili rimbalzi tra nodi adiacenti. - L'obstacle rettangolare (area rossa) rappresenta la zona senza nodi. - Le frecce mostrano il percorso che circumnaviga l'obstacle. - Quando si raggiunge un nodo (y, z) con distanza da D minore di quella del nodo di ingresso nel perimeter (x), si torna al greedy forwarding.

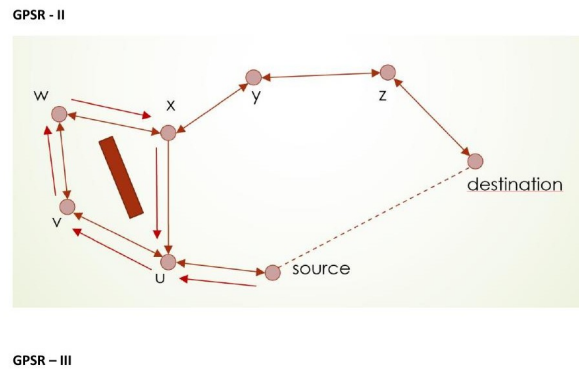


Figura 5.29. *Perimeter routing: il pacchetto naviga il confine del void (obstacle rettangolare) passando per $w \rightarrow x \rightarrow y \rightarrow z \rightarrow \text{destination}$, con alcuni nodi che rimandano indietro il pacchetto durante la face traversal.*

Importante: il perimeter routing funziona solo su un **grafo planare** (senza archi che si incrociano). Per questo GPSR richiede una fase di planarizzazione del grafo.

5.32 GPSR - III (Planarizzazione: Gabriel Graph e RNG)

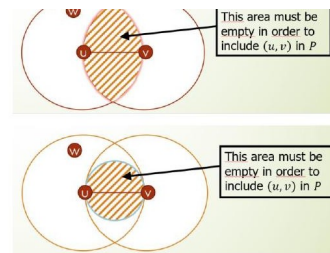


Figura 5.30. *Planarizzazione del grafo: GG (sopra) e RNG (sotto). Un arco (u,v) è incluso solo se nessun nodo w cade nell'area tratteggiata (cerchio per GG, lente per RNG).*

Per applicare il face routing, GPSR ha bisogno di un **grafo planare** (senza archi incrociati). Due algoritmi di planarizzazione distribuita sono usati:

Gabriel Graph (GG): - Un arco (u,v) è incluso nel GG se e solo se il **cerchio** con diametro $\overline{uv}$ non contiene nessun altro nodo w . - Formalmente: $|uw|^2 + |vw|^2 > |uv|^2$ per ogni w vicino di entrambi u e v . - Nel diagramma superiore: il cerchio (area tratteggiata) deve essere vuoto per includere l'arco.

Relative Neighborhood Graph (RNG): - Un arco (u, v) è incluso se la **regione a lente** (intersezione dei cerchi di raggio $|uv|$ centrati in u e v) non contiene nessun altro nodo w . - Formalmente: $|uw| < |uv|$ e $|vw| < |uv| \rightarrow w$ è nella lente $\rightarrow$ l'arco viene eliminato. - Nel diagramma inferiore: la lente tratteggiata deve essere vuota.

RNG GG: l'RNG è più sparso (meno archi) del GG, perché la lente è più piccola del cerchio. Entrambi producono grafi planari su cui il face routing funziona correttamente.

Distribuzione: ogni nodo può calcolare localmente quali archi includere o escludere, consultando solo i propri vicini diretti. Non è richiesto stato globale.

Nota – Riepilogo degli argomenti

Argomento	Lezione di riferimento
Interoperabilità, vertical silos, vendor lock-in	[[Lezione 3 - Interoperabilità e Standard]]
IoT Security, ITU-T Y.2066	[[Lezione 3 - Interoperabilità e Standard]]
MQTT pub/sub, topic, QoS 0/1/2	[[Lezione 4 - MQTT Part 1]]
MQTT sessioni, retained, last will, keepalive	[[Lezione 8 - MQTT Part 2]]
ZigBee join, stack, topologie, indirizzamento	[[Lezione 9 - The ZigBee Standard Part 1]]
ZigBee application layer, cluster, binding	[[Lezione 12 - Application Layer of ZigBee]]
Duty cycle, embedded programming Arduino/TinyOS	[[Lezione 23 - Embedded Programming e Arduino]]
SMAC, BMAC, X-MAC	[[Lezione 17 - MAC Protocols]]
IEEE 802.15.4 superframe, indirect transfer	[[Lezione 18 - The IEEE 802.15.4 Standard]]
Energy harvesting, architetture, fonti	[[Lezione 24 - Energy Harvesting IoT]]
Kansal, task model, energy neutrality	[[Lezione 28 - Kansal Problem e Energy Neutrality]]